


Rutgers University

# Trivial in Yokohama

Mateusz Klosin, Rishikesh Anabathula, Timothy Wu

ICPC Greater New York Regional

November 8, 2026

1 Contest 1
2 Mathematics 1
3 Data structures 2
4 Numerical 5
5 Number theory 8
6 Combinatorial 10
7 Graph 11
8 Geometry 16
9 Strings 21
10 Various 23

# Contest (1)

template.cpp — 18 lines

```
#include <bits/stdc++.h>
using namespace std;

#define f first
#define s second
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()

using ll = long long;
using pii = pair<int, int>;
using vi = vector<int>;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin.exceptions(cin.failbit);
}
```

baursakmake — 2 lines

```
export CXXFLAGS='-std=gnu++23 -g -Wall -Wextra -fsanitize=
    undefined,address'
export CXXFLAGS='-std=gnu++23 -O2'
```

run.sh — 1 lines

```
make $1 && ./$1 < bin/in.txt
```

samps.sh — 6 lines

```
make $1
for i in ~/Desktop/samps/$1*/sample/*.in; do
    ./$1 < $i > bin/out.txt
    echo $i ": "
    diff -Z ${i%.in}.ans bin/out.txt
done
```

checklist.txt — 5 lines

```
Template Typer Tasks:
1. Type template.cpp, baursakmake, run.sh, samps.sh
2. chmod +x run.sh samps.sh
3. Create bin/in.txt
4. Run the first line of baursakmake
```

xmodmap.sh — 11 lines

```
# find keycode
xev | grep keycode

# view key names
less /usr/include/X11/keysymdef.h

# remap key
xmodmap -e "keycode <number> = <name>"

# example
xmodmap -e "keycode 94 = Shift_L"
```

hash.sh — 3 lines

```
# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum |cut -c-6
```

test.sh — 26 lines

```
echo starting > bin/out1.txt
echo starting > bin/out2.txt
EXE=$1
C="${1}correct"
make $EXE
make $C
while diff -i bin/out1.txt bin/out2.txt; do
    python3 gen.py > bin/in.txt
    ./$C < bin/in.txt > bin/out1.txt
    ./$EXE < bin/in.txt > bin/out2.txt
    echo "OK"
done
echo "FAIL"
echo

echo "input:"
cat bin/in.txt
echo

echo "correct:"
cat bin/out1.txt
echo

echo "main:"
cat bin/out2.txt
echo
```

stress.sh — 7 lines

```
EXE=$1
make $EXE
python3 gen.py > bin/in.txt
while ./$EXE < bin/in.txt; do
    echo "OK"
    python3 gen.py > bin/in.txt
done
```

troubleshoot.txt — 52 lines

```
Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it.

Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?
```

# Mathematics (2)

## 2.1 Recurrences

If $a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k}$, and $r_1, \ldots, r_k$ are distinct roots of $x^k - c_1 x^{k-1} - \cdots - c_k$, there are $d_1, \ldots, d_k$ s.t.

$$a_n = d_1 r_1^n + \cdots + d_k r_k^n.$$

Non-distinct roots $r$ become polynomial factors, e.g. $a_n = (d_1 n + d_2) r^n$.

## 2.2 Geometry

### 2.2.1 Triangles

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2+2c^2-a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc\left[1-\left(\frac{a}{b+c}\right)^2\right]}$$

Law of tangents: $\frac{a+b}{a-b} = \frac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

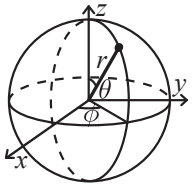
### 2.2.2 Quadrilaterals

With side lengths $a, b, c, d$, diagonals $e, f$, diagonals angle $\theta$, area $A$ and magic flux $F = b^2+d^2-a^2-c^2$:

$$4A = 2ef\cdot\sin\theta = F\tan\theta = \sqrt{4e^2f^2-F^2}$$

For cyclic quadrilaterals the sum of opposite angles is $180°$, $ef = ac+bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

### 2.2.3 Spherical coordinates




$$\begin{array}{cc} x = r\sin\theta\cos\phi & r = \sqrt{x^2+y^2+z^2} \\ y = r\sin\theta\sin\phi & \theta = \operatorname{acos}(z/\sqrt{x^2+y^2+z^2}) \\ z = r\cos\theta & \phi = \operatorname{atan2}(y,x) \end{array}$$

## 2.3 Derivatives/Integrals

$$\frac{d}{dx}\arcsin x = \frac{1}{\sqrt{1-x^2}} \qquad \frac{d}{dx}\arccos x = -\frac{1}{\sqrt{1-x^2}}$$

$$\frac{d}{dx}\tan x = 1+\tan^2 x \qquad \frac{d}{dx}\arctan x = \frac{1}{1+x^2}$$

$$\int \tan ax = -\frac{\ln|\cos ax|}{a} \qquad \int x\sin ax = \frac{\sin ax - ax\cos ax}{a^2}$$

$$\int e^{-x^2} = \frac{\sqrt{\pi}}{2}\operatorname{erf}(x) \qquad \int xe^{ax}dx = \frac{e^{ax}}{a^2}(ax-1)$$

## 2.4 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1}-c^a}{c-1}, c \neq 1$$

$$1^2+2^2+3^2+\cdots+n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3+2^3+3^3+\cdots+n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4+2^4+3^4+\cdots+n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

## 2.5 Polynomial interpolation

$$P(x) = \prod_{j=0}^{n-1}(x-x_j)$$

$$P'(x_i) = \prod_{j\neq i}(x_i-x_j)$$

For $n$ points $(x_i, y_i)$ with distinct $x_i$, the unique polynomial of degree at most $n-1$ passing through them is

$$p(x) = \sum_{i=0}^{n-1} y_i \prod_{j\neq i}\frac{x-x_j}{x_i-x_j} = \sum_{i=0}^{n-1}\frac{y_i}{P'(x_i)}\prod_{j\neq i}(x-x_j)$$

## 2.6 Distributions

### 2.6.1 Discrete distributions

#### Binomial distribution

The number of successes in $n$ independent yes/no experiments, each which yields success with probability $p$ is $\operatorname{Bin}(n,p)$, $n = 1,2,\ldots$, $0 \le p \le 1$.

$$p(k) = \binom{n}{k}p^k(1-p)^{n-k}$$

$$\mu = np,\ \sigma^2 = np(1-p)$$

$\operatorname{Bin}(n,p)$ is approximately $\operatorname{Po}(np)$ for small $p$.

#### First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability $p$ is $\operatorname{Fs}(p)$, $0 \le p \le 1$.

$$p(k) = p(1-p)^{k-1},\ k = 1,2,\ldots$$

$$\mu = \frac{1}{p},\ \sigma^2 = \frac{1-p}{p^2}$$

#### Poisson distribution

The number of events occurring in a fixed period of time $t$ if these events occur with a known average rate $\kappa$ and independently of the time since the last event is $\operatorname{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda}\frac{\lambda^k}{k!}, k = 0,1,2,\ldots$$

$$\mu = \lambda,\ \sigma^2 = \lambda$$

### 2.6.2 Continuous distributions

#### Exponential distribution

The time between events in a Poisson process is $\operatorname{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \ge 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda},\ \sigma^2 = \frac{1}{\lambda^2}$$

#### Normal distribution

Most real random values with mean $\mu$ and variance $\sigma^2$ are well described by $\mathcal{N}(\mu,\sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1,\sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2,\sigma_2^2)$ are independent, then

$$aX_1 + bX_2 + c \sim \mathcal{N}(a\mu_1+b\mu_2+c, a^2\sigma_1^2+b^2\sigma_2^2)$$

# Data structures (3)

## OrderStatisticTree.h

**Description:** A set (not multiset!) with support for finding the n'th element, and finding the index of an element. To get a map, change `null_type`.
**Time:** $\mathcal{O}(\log N)$

1a6cfa, 16 lines

```
#include <bits/extc++.h>
using namespace __gnu_pbds;

template<class T>
using Tree = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>; ↑cd2981

void example() {
  Tree<int> t, t2; t.insert(8);
  auto it = t.insert(10).f; ↑073c83
  assert(it == t.lower_bound(9));
  assert(t.order_of_key(10) == 1);
  assert(t.order_of_key(11) == 2);
  assert(*t.find_by_order(0) == 8);
  t.join(t2); // assuming T< T2 or T> T2, merge t2 into t
} ↑1a6cfa
```

## HashMap.h

**Description:** Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

d77092, 7 lines

```
#include <bits/extc++.h>
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
  const uint64_t C = ll(4e18 * acos(0)) | 71;
  ll operator()(ll x) const { return __builtin_bswap64(x*C); }
};
__gnu_pbds::gp_hash_table<ll,int,chash> h({},{},{},{},{1<<16});
```

## SegmentTree.h

**Description:** Zero-indexed sum-tree with point assignment and range sum queries. Bounds are inclusive on both ends: $[l, r]$. Empty if $l > r$. Can be changed by modifying T, f and id.
**Usage:** `seg tree(n); tree.modify(i, value); auto result = tree.query(l, r);`
**Time:** $\mathcal{O}(\log N)$

222c12, 19 lines

```
struct seg {
  using T = ll;
  T id=0;
  T f(T a, T b) {return a+b;}
  ll n; ↑20efa7
  vector<T> t;
  seg(ll n) : n(n), t(2*n, id) {}
  void modify(ll p, T value) {  // set value at position p
    for (p+=n, t[p] = value; p /= 2;) t[p] = f(t[2*p], t[2*p
        +1]);
  } ↑ee8f3f
  T query(ll l, ll r) { // fold f on interval [l, r]
    T resl=id, resr=id;
    for (l += n, r += n + 1; l < r; l /= 2, r /= 2) {
      if (l&1) resl = f(resl, t[l++]);
      if (r&1) resr = f(t[--r], resr);
    } ↑43b009
    return f(resl, resr);
  }
}; ↑222c12
```

## LazySegmentTree.h

**Description:** Zero-indexed iterative lazy segment tree on inclusive intervals $[l, r]$. Customize f, g, h, and the identities for the desired operations.
**Usage:** `lzseg tree(n); tree.modify(i, value); tree.modify(l, r, update); auto result = tree.query(l, r);`
**Time:** $\mathcal{O}(\log N)$.

6a8e5c, 64 lines

```
struct lzseg {
  typedef ll T;
  typedef ll U;
  T idT = 0;
  U idU = 0; ↑cd330d
  ll n, lg = 0;
  vector<T> t;
  vector<U> d;
  lzseg(ll n) : n(n), t(2*n, idT), d(n, idU) {
    for (ll i = 1; i < n; i *= 2) ++lg;
  } ↑ad84d2
  // combining segtree nodes a and b
  T f(T a, T b) { return a + b; }
  // applying updates a and b (in that order)
  U g(U b, U a) { return a + b; } ↑4bdb15
  // applying update b to segtree node a
  T h(U b, T a) { return a + b; }
  void calc(ll p) { t[p] = h(d[p], f(t[p * 2], t[p * 2 + 1]));
      }
  void apply(ll p, U v) {
    t[p] = h(v, t[p]); ↑1b5804
    if(p < n) d[p] = g(v, d[p]);
  }
  void push(ll p) {
    p += n; ↑bd3965
    for (ll s = lg; s > 0; --s) {
      ll i = p >> s;
      if(i && d[i] != idU) {
        apply(i * 2, d[i]);
        apply(i * 2 + 1, d[i]);
        d[i] = idU;
      } ↑b89747
    }
  }
  void modify(ll p, T v) {
    push(p); ↑747849
    t[p += n] = v;
    while(p > 1) calc(p /= 2);
  } ↑fa9694
  void modify(ll l, ll r, U v) {
    if (l > r) return;
    push(l), push(r);
    bool cl = false, cr = false;
    for(l += n, r += n + 1; l < r; l /= 2, r /= 2) {
      if(cl) calc(l - 1);
      if(cr) calc(r); ↑595833
      if(l & 1) apply(l++, v), cl = true;
      if(r & 1) apply(--r, v), cr = true;
    } ↑c6b877
    for(--l; r; l /= 2, r /= 2) {
      if(cl) calc(l);
      if(cr) calc(r);
    }
  } ↑8aef7f
  T query(ll l, ll r) {
    if (l > r) return idT;
    push(l), push(r);
    T resl = idT, resr = idT;
    for(l += n, r += n + 1; l < r; l /= 2, r /= 2) {
      if(l & 1) resl = f(resl, t[l++]);
      if(r & 1) resr = f(t[--r], resr);
    } ↑f717a7
    return f(resl, resr);
  }
}; ↑6a8e5c
```

## KlzSeg.h

**Description:** Segment tree with ability to add or set values of large intervals, and compute max of intervals. All constructor, query, and update bounds are inclusive $[l, r]$. Can be changed to other things. Use with a bump allocator for better performance, and SmallPtr or implicit indices to save memory.
**Usage:** `Node* tr = new Node(v, 0, sz(v)-1);`
**Time:** $\mathcal{O}(\log N)$.

"../various/BumpAllocator.h" d00570, 50 lines

```
const int inf = 1e9;
struct Node {
  Node *l = 0, *r = 0;
  int lo, hi, mset = inf, madd = 0, val = -inf; ↑d01722
  Node(int lo,int hi):lo(lo),hi(hi){} // Large interval of -inf
  Node(vi& v, int lo, int hi) : lo(lo), hi(hi) {
    if (lo < hi) {
      int mid = lo + (hi - lo)/2;
      l = new Node(v, lo, mid); r = new Node(v, mid+1, hi);
      val = max(l->val, r->val);
    } ↑b9a526
    else val = v[lo];
  }
  int query(int L, int R) {
    if (R < lo || hi < L) return -inf; ↑c3819f
    if (L <= lo && hi <= R) return val;
    push();
    return max(l->query(L, R), r->query(L, R));
  } ↑e0ef37
  void set(int L, int R, int x) {
    if (R < lo || hi < L) return;
    if (L <= lo && hi <= R) mset = val = x, madd = 0;
    else {
      push(), l->set(L, R, x), r->set(L, R, x);
      val = max(l->val, r->val);
    } ↑53f612
  }
  void add(int L, int R, int x) {
    if (R < lo || hi < L) return; ↑5eb062
    if (L <= lo && hi <= R) {
      if (mset != inf) mset += x;
      else madd += x;
      val += x;
    } ↑6fd183
    else {
      push(), l->add(L, R, x), r->add(L, R, x);
      val = max(l->val, r->val);
    }
  } ↑69d825
  void push() {
    if (!l) {
      int mid = lo + (hi - lo)/2;
      l = new Node(lo, mid); r = new Node(mid+1, hi);
    } ↑f8c579
    if (mset != inf)
      l->set(lo,hi,mset), r->set(lo,hi,mset), mset = inf;
    else if (madd)
      l->add(lo,hi,madd), r->add(lo,hi,madd), madd = 0;
  }
}; ↑d00570
```

## GennadyKeshstein.h

**Description:** Finds the maximum right endpoint after a given left endpoint such that the interval satisfies a monotonic condition. Requires Node, st, and push(i). Bounds are inclusive; pr is the aggregate preceding this segment.
**Time:** $\mathcal{O}(\log N)$

9c663d, 18 lines

```
pair<int, Node> walk(int le, function<bool(Node)> &pred,
                     Node pr, int i, int l, int r) {
  if (le == l) {
    Node v = pr + st[i];
    if (pred(v))
      return {r, v};
  } ↑97fc3d
  if (l == r)
    return {l - 1, pr};
  push(i); ↑fffced
  int m = (l + r) / 2;
  if (le > m)
    return walk(le, pred, pr, i << 1 | 1, m + 1, r);
  auto p = walk(le, pred, pr, i << 1, l, m); ↑86f194
  if (p.f == m)
    p = walk(m + 1, pred, p.s, i << 1 | 1, m + 1, r);
  return p;
} ↑9c663d
```

## UnionFindRollback.h

**Description:** Disjoint-set data structure with undo. If undo is not needed, skip st, time() and rollback().
**Usage:** `int t = uf.time(); ...; uf.rollback(t);`
**Time:** $\mathcal{O}(\log(N))$

9633e7, 21 lines

```
struct RollbackUF {
  vi e; vector<pii> st;
  RollbackUF(int n) : e(n, -1) {}
  int size(int x) { return -e[find(x)]; } ↑dfd9e1
  int find(int x) { return e[x] < 0 ? x : find(e[x]); }
  int time() { return sz(st); }
  void rollback(int t) {
    for (int i = time(); i --> t;)
      e[st[i].f] = st[i].s;
    st.resize(t);
  } ↑584096
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b); ↑9e55d4
```

```
    st.push_back({a, e[a]});
    st.push_back({b, e[b]});
    e[a] += e[b]; e[b] = a;
    return true;
  }
}; ↑9633e7
```

## LineContainer.h

**Description:** Container where you can add lines of the form $kx + m$, and query maximum values at points $x$. Useful for dynamic programming ("convex hull trick").
**Time:** $\mathcal{O}(\log N)$

8ec1c7, 30 lines

```
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
}; ↑7e3ecf

struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const ll inf = LLONG_MAX; ↑a3ffb4
  ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k); ↑d6d755
    return x->p >= y->p;
  }
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z); ↑e189b8
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  } ↑4e2c33
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
}; ↑8ec1c7
```

## CHT.h

**Description:** Add lines $kx + m$ and query maximum values (minimum if `queryMax=false`). Slopes must be nondecreasing for max, nonincreasing for min; query points must be nondecreasing. Queries require at least one line. Coefficient negation and evaluation arithmetic must fit in ll, and cross products in signed __int128.
**Time:** $\mathcal{O}(1)$ amortized per operation.

<cassert> e14a38, 33 lines

```
struct CHT {
  struct Line {
    ll k, m;
    ll eval(ll x) const { return k*x + m; }
  }; ↑5201ab
  deque<Line> q;
  ll last = LLONG_MIN;
  int sign; ↑241d74
  CHT(bool queryMax = true) : sign(queryMax ? 1 : -1) {}
  bool bad(Line a, Line b, Line c) {
    return ((__int128)a.m - b.m) * ((__int128)c.k - b.k)
      >= ((__int128)b.m - c.m) * ((__int128)b.k - a.k);
  } ↑29f4c9
  void add(ll k, ll m) {
    k *= sign; m *= sign;
    assert(q.empty() || q.back().k <= k);
    if (!q.empty() && q.back().k == k) {
      if (q.back().m >= m) return;
      q.pop_back();
    } ↑8bb31b
    Line l{k, m};
    while (sz(q) >= 2 && bad(q[sz(q)-2], q.back(), l))
      q.pop_back();
    q.push_back(l);
  } ↑371c5e
  ll query(ll x) {
    assert(!q.empty() && x >= last);
    last = x;
    while (sz(q) >= 2 && q[0].eval(x) <= q[1].eval(x))
      q.pop_front(); ↑86dcc9
    return sign * q[0].eval(x);
  }
}; ↑e14a38
```

## Treap.h

**Description:** A short self-balancing tree. It acts as a sequential container with log-time splits/joins, and is easy to augment with additional data.
**Time:** $\mathcal{O}(\log N)$

b7bfcb, 54 lines

```
struct Node {
  Node *l = 0, *r = 0;
  int val, y, c = 1;
  Node(int val) : val(val), y(rand()) {}
  void recalc();
}; ↑daabb7

int cnt(Node* n) { return n ? n->c : 0; }
void Node::recalc() { c = cnt(l) + cnt(r) + 1; } ↑371cf9

template<class F> void each(Node* n, F f) {
  if (n) { each(n->l, f); f(n->val); each(n->r, f); }
} ↑cfbf7f

// Split by count: first k elements and the rest (k is not an
    endpoint).
pair<Node*, Node*> split(Node* n, int k) {
  if (!n) return {};
  if (cnt(n->l) >= k) { // "n->val >= k" for lower_bound(k)
    auto [L,R] = split(n->l, k);
    n->l = R; ↑d0f96d
    n->recalc();
    return {L, n};
  } else {
    auto [L,R] = split(n->r,k - cnt(n->l) - 1); // and just "k"
    n->r = L; ↑b25feb
    n->recalc();
    return {n, R};
  }
} ↑b242de

Node* merge(Node* l, Node* r) {
  if (!l) return r;
  if (!r) return l;
  if (l->y > r->y) {
    l->r = merge(l->r, r); ↑67d816
    return l->recalc(), l;
  } else {
    r->l = merge(l, r->l);
    return r->recalc(), r;
  } ↑d588a0
}

Node* ins(Node* t, Node* n, int pos) {
  auto [l,r] = split(t, pos);
  return merge(merge(l, n), r);
} ↑47352e

// Move [l, r) before original index k (k <= l or k > r).
void move(Node*& t, int l, int r, int k) {
  Node *a, *b, *c; ↑dcf85b
  tie(a,b) = split(t, l); tie(b,c) = split(b, r - l + 1);
  if (k <= l) t = merge(ins(a, b, k), c);
  else t = merge(a, ins(c, b, k - r - 1));
} ↑b7bfcb
```

## FenwickTree.h

**Description:** Computes partial sums $a[0] + a[1] + \cdots + a[\text{pos}]$, and updates single elements $a[i]$, taking the difference between the old and new value.
**Time:** Both operations are $\mathcal{O}(\log N)$.

d03b13, 22 lines

```
struct FT {
  vector<ll> s;
  FT(int n) : s(n) {}
  void update(int pos, ll dif) { // a[pos] += dif
    for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
  } ↑a0c54f
  ll query(int pos) { // sum of [0, pos); query(-1) = 0
    ll res = 0;
    for (++pos; pos > 0; pos &= pos - 1) res += s[pos-1];
    return res;
  } ↑9e085c
  int lower_bound(ll sum) {// min pos st sum of [0, pos] >= sum
    // Requires nonnegative values. Returns n if no sum is >=
        sum, or -1 if empty sum is.
    if (sum <= 0) return -1;
    int pos = 0; ↑0b4bb7
    for (int pw = 1 << 25; pw; pw >>= 1) {
      if (pos + pw <= sz(s) && s[pos + pw-1] < sum)
        pos += pw, sum -= s[pos-1];
    }
    return pos;
  }
}; ↑d03b13
```

## FenwickTree2D.h

**Description:** Computes sums $a[i, j]$ for all $i \le I$, $j \le J$, and increases single elements $a[i, j]$. Requires that the elements to be updated are known in advance (call fakeUpdate() before init()).
**Time:** $\mathcal{O}(\log^2 N)$. (Use persistent segment trees for $\mathcal{O}(\log N)$.)

"FenwickTree.h" 6f0f5d, 24 lines

```
struct FT2 {
  vector<vi> ys; vector<FT> ft;
  FT2(int limx) : ys(limx) {}
  void fakeUpdate(int x, int y) {
    for (; x < sz(ys); x |= x + 1) ys[x].push_back(y);
  } ↑fe99f8
  void init() {
    for (vi& v : ys) sort(all(v)), ft.emplace_back(sz(v));
  } ↑1e4363
  int ind(int x, int y) {
    return (int)(lower_bound(all(ys[x]), y) - ys[x].begin()); }
  void update(int x, int y, ll dif) {
    for (; x < sz(ys); x |= x + 1)
      ft[x].update(ind(x, y), dif);
  } ↑c48166
  ll query(int x, int y) { // inclusive upper bounds; x = -1 is
      empty
    ll sum = 0;
    for (++x; x; x &= x - 1) {
      int p = (int)(upper_bound(all(ys[x-1]), y) - ys[x-1].
          begin());
      sum += ft[x-1].query(p-1);
    } ↑f3d370
    return sum;
  }
```

```
};  ↑6f0f5d
```

### RMQ.h

**Description:** Range Minimum Queries on an array. Returns $\min(V[a], V[a+1], \ldots, V[b])$ in constant time.
**Usage:** `RMQ rmq(values);`
`rmq.query(l, r); // inclusive [l, r]`
**Time:** $\mathcal{O}(|V| \log |V| + Q)$

d74d4f, 16 lines

```
template<class T>
struct RMQ {
  vector<vector<T>> jmp;
  RMQ(const vector<T>& V) : jmp(1, V) {
    for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
      jmp.emplace_back(sz(V) - pw * 2 + 1);  ↑7420f3
      rep(j,0,sz(jmp[k]))
        jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
    }
  }  ↑dffd89
  T query(int a, int b) {
    assert(a <= b); // empty ranges are not supported
    int dep = 31 - __builtin_clz(b - a + 1);
    return min(jmp[dep][a], jmp[dep][b + 1 - (1 << dep)]);
  }
};  ↑d74d4f
```

### MoQueries.h

**Description:** Answer inclusive $[l, r]$ interval or tree path queries by finding an approximate TSP through the queries and moving between queries by adding/removing points at the ends. Tree paths include both endpoints. Implement add/del/calc below. Use $[l, l-1]$ for an empty array query. If values are on tree edges, change `step` to add/remove the edge $(a, c)$ and remove the initial `add` call (but keep `in`).
**Time:** $\mathcal{O}\left(N\sqrt{Q}\right)$

969be6, 50 lines

```
void add(int ind, int end); // add a[ind] (end = 0 or 1)
void del(int ind, int end); // remove a[ind]
int calc(); // compute current answer

vi mo(vector<pii> Q) {
  int L = 0, R = -1, blk = 350; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s;  ↑d34a32
#define K(x) pii(x.f/blk, x.s ^ -(x.f/blk & 1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
  for (int qi : s) {
    pii q = Q[qi];  ↑43c212
    while (L > q.f) add(--L, 0);
    while (R < q.s) add(++R, 1);
    while (L < q.f) del(L++, 0);  ↑791d77
    while (R > q.s) del(R--, 1);
    res[qi] = calc();
  }
  return res;
#undef K
}  ↑8cb64c

vi moTree(vector<array<int, 2>> Q, vector<vi>& ed, int root=0){
  int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
  add(0, 0), in[0] = 1;  ↑31b8c3
  auto dfs = [&](int x, int p, int dep, auto& f) -> void {
    par[x] = p;
    L[x] = N;
    if (dep) I[x] = N++;  ↑ad98d7
    for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
    if (!dep) I[x] = N++;
    R[x] = N-1;
  };  ↑f3438c
  dfs(root, -1, 0, dfs);
#define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk & 1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
  for (int qi : s) rep(end,0,2) {
    int &a = pos[end], b = Q[qi][end], i = 0;  ↑1e410b
#define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
                  else { add(c, end); in[c] = 1; } a = c; }
    while (!(L[b] <= L[a] && R[a] <= R[b]))
      I[i++] = b, b = par[b];
    while (a != b) step(par[a]);  ↑dc40dd
    while (i--) step(I[i]);
    if (end) res[qi] = calc();
  }
  return res;
}  ↑969be6
```

# Numerical (4)

## 4.1 Summation

### NeumaierSum.h

**Description:** More precise summation for floating point types. Absolute error at most $\epsilon|\sum a_i|/2 + (3n^2/16 + n/4)\epsilon^2 \sum |a_i|$. Sum must be finite and not overflow. Do not use with `Ofast`. Use only in desparate situations.
**Time:** $\mathcal{O}(n)$

c84e23, 12 lines

```
template <class T> T neumaierSum(const vector<T> &a) {
  T sum = 0, c = 0;
  for (T x : a) {
    T t = sum + x;  ↑448005
    if (abs(sum) >= abs(x))
      c += (sum - t) + x;
    else
      c += (x - t) + sum;
    sum = t;
  }
  return sum + c;
}  ↑c84e23
```

## 4.2 Polynomials and recurrences

### Polynomial.h

c9b7b0, 17 lines

```
struct Poly {
  vector<double> a;
  double operator()(double x) const {
    double val = 0;
    for (int i = sz(a); i--;) (val *= x) += a[i];  ↑1b799c
    return val;
  }
  void diff() {
    rep(i,1,sz(a)) a[i-1] = i*a[i];
    a.pop_back();
  }  ↑d447a3
  void divroot(double x0) {
    double b = a.back(), c; a.back() = 0;
    for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c;
    a.pop_back();
  }
};  ↑c9b7b0
```

### PolyRoots.h

**Description:** Finds the real roots to a polynomial.
**Usage:** `polyRoots({{2,-3,1}},-1e9,1e9) // solve x^2-3x+2 = 0`
**Time:** $\mathcal{O}\left(n^2 \log(1/\epsilon)\right)$

"Polynomial.h" b00bfe, 23 lines

```
vector<double> polyRoots(Poly p, double xmin, double xmax) {
  if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
  vector<double> ret;
  Poly der = p;
  der.diff();  ↑8409d9
  auto dr = polyRoots(der, xmin, xmax);
  dr.push_back(xmin-1);
  dr.push_back(xmax+1);
  sort(all(dr));  ↑5604f0
  rep(i,0,sz(dr)-1) {
    double l = dr[i], h = dr[i+1];
    bool sign = p(l) > 0;
    if (sign ^ (p(h) > 0)) {
      rep(it,0,60) { // while (h - l > 1e-8)
        double m = (l + h) / 2, f = p(m);  ↑40bd6f
        if ((f <= 0) ^ sign) l = m;
        else h = m;
      }
      ret.push_back((l + h) / 2);
    }  ↑1c9b1d
  }
  return ret;
}  ↑b00bfe
```

### PolyInv.h

**Description:** Returns $1/a$ modulo $x^n$. Requires $a[0] \neq 0$, $n \geq 0$, coefficients in $[0, \text{mod})$.
**Time:** $\mathcal{O}(n \log n)$

"NumberTheoreticTransform.h" 280ceb, 15 lines

```
vl polyInv(const vl& a, int n) {
  if (!n) return {};
  vl b{modpow(a[0], mod - 2)};
  while (sz(b) < n) {
    int m = min(2 * sz(b), n);  ↑7264c8
    vl c(a.begin(), a.begin() + min(sz(a), m));
    c = conv(c, b);
    c.resize(m);
    for (ll& x : c) x = (mod - x) % mod;
    c[0] = (c[0] + 2) % mod;  ↑c29b4b
    b = conv(b, c);
    b.resize(m);
  }
  return b;
}  ↑280ceb
```

### Newton.h

**Description:** Solves $F(a) = 0$ modulo $x^n$, starting from a constant root $a_0$. f(a, m) returns a pair $(F(a), F'(a))$ modulo $x^m$; derivative is w.r.t. $a$. Requires $F'(a_0)[0] \neq 0$, $n \geq 0$, coefficients in $[0, \text{mod})$. Each step uses $a \leftarrow a - F(a)/F'(a)$.
**Time:** $\mathcal{O}(n \log n)$ if f takes $\mathcal{O}(m \log m)$

"PolyInv.h" e8671f, 14 lines

```
template<class F>
vl newton(ll a0, int n, F f) {
  if (!n) return {};
  vl a{a0};  ↑a8e134
  while (sz(a) < n) {
    int m = min(2 * sz(a), n);
    auto [b, c] = f(a, m);
    b.resize(m);
    b = conv(b, polyInv(c, m));
    a.resize(m);  ↑496951
    rep(i,0,m) a[i] = (a[i] - b[i] + mod) % mod;
  }
  return a;
}  ↑e8671f
```

## BerlekampMassey.h

**Description:** Recovers any $n$-order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.
**Usage:** `berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}`
**Time:** $\mathcal{O}(N^2)$

"../number-theory/ModPow.h" 96548b, 20 lines

```
vector<ll> berlekampMassey(vector<ll> s) {
  int n = sz(s), L = 0, m = 0;
  vector<ll> C(n), B(n), T;
  C[0] = B[0] = 1;  ↑b21e6e

  ll b = 1;
  rep(i,0,n) { ++m;
    ll d = s[i] % mod;
    rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
    if (!d) continue;  ↑f1ebd1
    T = C; ll coef = d * modpow(b, mod-2) % mod;
    rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
    if (2 * L > i) continue;
    L = i + 1 - L; B = T; b = d; m = 0;
  }  ↑3dc38b

  C.resize(L + 1); C.erase(C.begin());
  for (ll& x : C) x = (mod - x) % mod;
  return C;
}  ↑96548b
```

## LinearRecurrence.h

**Description:** Generates the $k$'th term of an $n$-order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \ldots \geq n-1]$ and $tr[0 \ldots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey.
**Usage:** `linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number`
**Time:** $\mathcal{O}(n^2 \log k)$

f4e444, 26 lines

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
  int n = sz(tr);

  auto combine = [&](Poly a, Poly b) {
    Poly res(n * 2 + 1);  ↑d3cd51
    rep(i,0,n+1) rep(j,0,n+1)
      res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
    for (int i = 2 * n; i > n; --i) rep(j,0,n)
      res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
    res.resize(n + 1);  ↑697752
    return res;
  };

  Poly pol(n + 1), e(pol);  ↑5d89c0
  pol[0] = e[1] = 1;

  for (++k; k; k /= 2) {
    if (k % 2) pol = combine(pol, e);
    e = combine(e, e);
  }  ↑b658e4

  ll res = 0;
  rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
  return res;
}  ↑f4e444
```

## BostanMori.h

**Description:** Returns $[x^k]P(x)/Q(x)$. Requires $Q[0] \neq 0$, $k \geq 0$, coefficients in $[0, \text{mod})$. For a recurrence with initial terms $S$ and coefficients $c$, use $Q = \{1, -c_0, -c_1, \ldots\}$ and $P = SQ \bmod x^{|S|}$ (normalize coefficients).

**Time:** $\mathcal{O}(d \log d \log(k+1))$, $d = \max(|P|, |Q|)$

"NumberTheoreticTransform.h" bd30e2, 17 lines

```
ll bostanMori(vl p, vl q, ll k) {
  for (; k; k /= 2) {
    vl r = q;
    for (int i = 1; i < sz(r); i += 2)
      r[i] = (mod - r[i]) % mod;  ↑ee0c60
    p = conv(p, r);
    q = conv(q, r);
    int n = 0;
    for (int i = int(k & 1); i < sz(p); i += 2)
      p[n++] = p[i];  ↑4a78ce
    p.resize(n);
    n = 0;
    for (int i = 0; i < sz(q); i += 2) q[n++] = q[i];
    q.resize(n);
  }
  return p.empty() ? 0 : p[0] * modpow(q[0], mod-2) % mod;
}  ↑bd30e2
```

# 4.3 Optimization

## HillClimbing.h

**Description:** Poor man's optimization for unimodal functions.

546db4, 14 lines

```
typedef array<double, 2> P;

template<class F> pair<double, P> hillClimb(P start, F f) {
  pair<double, P> cur(f(start), start);  ↑18b365
  for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
    rep(j,0,100) rep(dx,-1,2) rep(dy,-1,2) {
      P p = cur.s;
      p[0] += dx*jmp;
      p[1] += dy*jmp;
      cur = min(cur, make_pair(f(p), p));
    }  ↑0415cc
  }
  return cur;
}  ↑546db4
```

## Integrate.h

**Description:** Simple integration of a function over an interval using Simpson's rule. The error should be proportional to $h^4$, although in practice you will want to verify that the result is stable to desired precision when epsilon changes.

4756fc, 7 lines

```
template<class F>
double quad(double a, double b, F f, const int n = 1000) {
  double h = (b - a) / 2 / n, v = f(a) + f(b);
  rep(i,1,n*2)
    v += f(a + i*h) * (i&1 ? 4 : 2);
  return v * h / 3;
}  ↑4756fc
```

## IntegrateAdaptive.h

**Description:** Fast integration using an adaptive Simpson's rule.
**Usage:** `double sphereVolume = quad(-1, 1, [](double x) {`
`return quad(-1, 1, [&](double y) {`
`return quad(-1, 1, [&](double z) {`
`return x*x + y*y + z*z < 1; });});});`

92dd79, 15 lines

```
typedef double d;
#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6

template <class F>
d rec(F& f, d a, d b, d eps, d S) {
  d c = (a + b) / 2;  ↑eda167
  d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
  if (abs(T - S) <= 15 * eps || b - a < 1e-10)
    return T + (T - S) / 15;
  return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2, S2);
}  ↑a81d9a
template<class F>
d quad(d a, d b, F f, d eps = 1e-8) {
  return rec(f, a, b, eps, S(a, b));
}  ↑92dd79
```

## Simplex.h

**Description:** Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \leq b$, $x \geq 0$. Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal $x$ (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.
**Usage:** `vvd A = {{1,-1}, {-1,1}, {-1,-2}};`
`vd b = {1,1,-4}, c = {-1,-1}, x;`
`T val = LPSolver(A, b, c).solve(x);`
**Time:** $\mathcal{O}(NM * \#pivots)$, where a pivot may be e.g. an edge relaxation. $\mathcal{O}(2^n)$ in the general case.

aa8530, 73 lines

```
typedef double T; // long double, Rational, double + mod<P>...
typedef vector<T> vd;
typedef vector<vd> vvd;

const T eps = 1e-8, inf = 1/.0;  ↑6296c1
#define MP make_pair
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s])) s=j

struct LPSolver {
  int m, n;  ↑34f6a6
  vi N, B;
  vvd D;

  LPSolver(const vvd& A, const vd& b, const vd& c) :
    m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
      rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];  ↑a00ca8
      rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] = b[i];}
      rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
      N[n] = -1; D[m+1][n] = 1;
    }  ↑dcadf8

  void pivot(int r, int s) {
    T *a = D[r].data(), inv = 1 / a[s];
    rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
      T *b = D[i].data(), inv2 = b[s] * inv;  ↑a86c76
      rep(j,0,n+2) b[j] -= a[j] * inv2;
      b[s] = a[s] * inv2;
    }  ↑df792b
    rep(j,0,n+2) if (j != s) D[r][j] *= inv;
    rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
    D[r][s] = inv;
    swap(B[r], N[s]);
  }  ↑193de8

  bool simplex(int phase) {
    int x = m + phase - 1;
    for (;;) {
      int s = -1;
      rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
      if (D[x][s] >= -eps) return true;  ↑e72781
      // Bland's rule (slower, guarantees no cycles)
      /* replace the 2 lines above with:
         rep(j,0,n+1) if (N[j] != -phase && D[x][j] < -eps)
             if (s == -1 || N[j] < N[s]) s = j;
         if (s == -1) return true; */
      int r = -1;  ↑fcd18c
      rep(i,0,m) {
        if (D[i][s] <= eps) continue;
        if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
```

```
                    < MP(D[r][n+1] / D[r][s], B[r])) r = i;
      } ↑170720
      if (r == -1) return false;
      pivot(r, s);
    }
  } ↑62b7d3

  T solve(vd &x) {
    int r = 0;
    rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i; ↑cc8cd8
    if (D[r][n+1] < -eps) {
      pivot(r, n);
      if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
      rep(i,0,m) if (B[i] == -1) {
        int s = 0;
        rep(j,1,n+1) ltj(D[i]); ↑db9144
        pivot(i, s);
      }
    } ↑36d5c1
    bool ok = simplex(1); x = vd(n);
    rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
    return ok ? D[m][n+1] : inf;
  }
}; ↑aa8530
```

## 4.4 Matrices

### Determinant.h

**Description:** Calculates determinant of a matrix. Destroys the matrix.
**Time:** $\mathcal{O}(N^3)$

bd5cec, 15 lines

```
double det(vector<vector<double>>& a) {
  int n = sz(a); double res = 1;
  rep(i,0,n) {
    int b = i; ↑4bd724
    rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
    if (i != b) swap(a[i], a[b]), res *= -1;
    res *= a[i][i];
    if (res == 0) return 0;
    rep(j,i+1,n) {
      double v = a[j][i] / a[i][i]; ↑356eb5
      if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
    }
  }
  return res;
} ↑bd5cec
```

### IntDeterminant.h

**Description:** Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.
**Time:** $\mathcal{O}(N^3)$

3313dc, 18 lines

```
const ll mod = 12345;
ll det(vector<vector<ll>>& a) {
  int n = sz(a); ll ans = 1; ↑aeac6f
  rep(i,0,n) {
    rep(j,i+1,n) {
      while (a[j][i] != 0) { // gcd step
        ll t = a[i][i] / a[j][i];
        if (t) rep(k,i,n)
          a[i][k] = (a[i][k] - a[j][k] * t) % mod;
        swap(a[i], a[j]); ↑4d6748
        ans *= -1;
      }
    } ↑7effce
    ans = ans * a[i][i] % mod;
    if (!ans) return 0;
  }
  return (ans + mod) % mod;
} ↑3313dc
```

### SolveLinear.h

**Description:** Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in $A$ and $b$ is lost.
**Time:** $\mathcal{O}(n^2 m)$

44c9ab, 38 lines

```
typedef vector<double> vd;
const double eps = 1e-12;

int solveLinear(vector<vd>& A, vd& b, vd& x) {
  int n = sz(A), m = sz(x), rank = 0, br, bc; ↑2cfbc7
  if (n) assert(sz(A[0]) == m);
  vi col(m); iota(all(col), 0);

  rep(i,0,n) {
    double v, bv = 0; ↑cdb1df
    rep(r,i,n) rep(c,i,m)
      if ((v = fabs(A[r][c])) > bv)
        br = r, bc = c, bv = v;
    if (bv <= eps) {
      rep(j,i,n) if (fabs(b[j]) > eps) return -1; ↑008896
      break;
    }
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]); ↑b1eb75
    rep(j,0,n) swap(A[j][i], A[j][bc]);
    bv = 1/A[i][i];
    rep(j,i+1,n) {
      double fac = A[j][i] * bv;
      b[j] -= fac * b[i]; ↑f8d04b
      rep(k,i+1,m) A[j][k] -= fac*A[i][k];
    }
    rank++;
  } ↑66cd8f

  x.assign(m, 0);
  for (int i = rank; i--;) {
    b[i] /= A[i][i];
    x[col[i]] = b[i];
    rep(j,0,i) b[j] -= A[j][i] * b[i]; ↑a0bd4f
  }
  return rank; // (multiple solutions if rank < m)
} ↑44c9ab
```

### SolveLinear2.h

**Description:** To get all uniquely determined values of $x$ back from SolveLinear, make the following changes:

"SolveLinear.h" 08e495, 7 lines

```
rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
// ... then at the end:
x.assign(m, undefined);
rep(i,0,rank) {
  rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
  x[col[i]] = b[i] / A[i][i];
fail:; } ↑08e495
```

### SolveLinearBinary.h

**Description:** Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys $A$ and $b$.
**Time:** $\mathcal{O}(n^2 m)$

fa2d7a, 34 lines

```
typedef bitset<1000> bs;

int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
  int n = sz(A), rank = 0, br;
  assert(m <= sz(x)); ↑d90d1b
  vi col(m); iota(all(col), 0);
  rep(i,0,n) {
    for (br=i; br<n; ++br) if (A[br].any()) break;
    if (br == n) {
      rep(j,i,n) if(b[j]) return -1; ↑f718ae
      break;
    }
    int bc = (int)A[br]._Find_next(i-1);
    swap(A[i], A[br]);
    swap(b[i], b[br]); ↑94782d
    swap(col[i], col[bc]);
    rep(j,0,n) if (A[j][i] != A[j][bc]) {
      A[j].flip(i); A[j].flip(bc);
    } ↑bf5e08
    rep(j,i+1,n) if (A[j][i]) {
      b[j] ^= b[i];
      A[j] ^= A[i];
    }
    rank++;
  } ↑4de1ff

  x = bs();
  for (int i = rank; i--;) {
    if (!b[i]) continue;
    x[col[i]] = 1; ↑c2244c
    rep(j,0,i) b[j] ^= A[j][i];
  }
  return rank; // (multiple solutions if rank < m)
} ↑fa2d7a
```

### MatrixInverse.h

**Description:** Invert matrix $A$. Returns rank; result is stored in $A$ unless singular (rank $< n$). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where $A^{-1}$ starts as the inverse of $A$ mod $p$, and $k$ is doubled in each step.
**Time:** $\mathcal{O}(n^3)$

ebfff6, 35 lines

```
int matInv(vector<vector<double>>& A) {
  int n = sz(A); vi col(n);
  vector<vector<double>> tmp(n, vector<double>(n));
  rep(i,0,n) tmp[i][i] = 1, col[i] = i; ↑9a9a66

  rep(i,0,n) {
    int r = i, c = i;
    rep(j,i,n) rep(k,i,n)
      if (fabs(A[j][k]) > fabs(A[r][c]))
        r = j, c = k; ↑6b4e10
    if (fabs(A[r][c]) < 1e-12) return i;
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
    swap(col[i], col[c]); ↑6ce940
    double v = A[i][i];
    rep(j,i+1,n) {
      double f = A[j][i] / v;
      A[j][i] = 0;
      rep(k,i+1,n) A[j][k] -= f*A[i][k]; ↑9da1ac
      rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
    }
    rep(j,i+1,n) A[i][j] /= v;
    rep(j,0,n) tmp[i][j] /= v;
    A[i][i] = 1;
  } ↑cd352a

  for (int i = n-1; i > 0; --i) rep(j,0,i) {
    double v = A[j][i];
    rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
  } ↑fd4d51

  rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
  return n;
} ↑ebfff6
```

### Tridiagonal.h

**Description:** $x = \text{tridiagonal}(d, p, q, b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i,\ 1 \le i \le n,$$

where $a_0$, $a_{n+1}$, $b_i$, $c_i$ and $d_i$ are known. $a$ can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, ..., -1, 1\}, \{0, c_1, c_2, \ldots, c_n\}, \{b_1, b_2, \ldots, b_n, 0\}, \{a_0, d_1, d_2, \ldots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.
If $|d_i| > |p_i| + |q_{i-1}|$ for all $i$, or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither `tr` nor the check for `diag[i] == 0` is needed.
**Time:** $\mathcal{O}(N)$

8f9fa8, 26 lines

```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T>& super,
    const vector<T>& sub, vector<T> b) {
  int n = sz(b); vi tr(n);  ↑52eb69
  rep(i,0,n-1) {
    if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] == 0
      b[i+1] -= b[i] * diag[i+1] / super[i];
      if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
      diag[i+1] = sub[i]; tr[++i] = 1;  ↑5606e8
    } else {
      diag[i+1] -= super[i]*sub[i]/diag[i];
      b[i+1] -= b[i]*sub[i]/diag[i];
    }
  }  ↑7da0d1
  for (int i = n; i--;) {
    if (tr[i]) {
      swap(b[i], b[i-1]);
      diag[i-1] = diag[i];
      b[i] /= super[i-1];  ↑6bd4e6
    } else {
      b[i] /= diag[i];
      if (i) b[i-1] -= b[i]*super[i-1];
    }
  }
  return b;
}  ↑8f9fa8
```

## 4.5 Fourier transforms

### FastFourierTransform.h

**Description:** fft(a) computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all $k$. $N$ must be a power of 2. Useful for convolution: `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by $n$, reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice $10^{16}$; higher for random inputs). Otherwise, use NTT/FFTMod.
**Time:** $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

00ced6, 35 lines

```
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vector<complex<long double>> R(2, 1);  ↑1ec777
  static vector<C> rt(2, 1);  // (^ 10% faster if double)
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n); rt.resize(n);
    auto x = polar(1.0L, acos(-1.0L) / k);
    rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
  }  ↑42ea68
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);  ↑8afdf7
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
      C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
      a[i + j + k] = a[i + j] - z;
      a[i + j] += z;
    }  ↑865e86
}
vd conv(const vd& a, const vd& b) {
  if (a.empty() || b.empty()) return {};
  vd res(sz(a) + sz(b) - 1);
  int L = 32 - __builtin_clz(sz(res)), n = 1 << L;  ↑ef397c
  vector<C> in(n), out(n);
  copy(all(a), begin(in));
  rep(i,0,sz(b)) in[i].imag(b[i]);
  fft(in);
  for (C& x : in) x *= x;  ↑e6330e
  rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
  fft(out);
  rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
  return res;
}  ↑00ced6
```

### FastFourierTransformMod.h

**Description:** Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice $10^{16}$ or higher). Inputs must be in $[0, \text{mod})$.
**Time:** $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

"FastFourierTransform.h" b82773, 22 lines

```
typedef vector<ll> vl;
template<int M> vl convMod(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  vl res(sz(a) + sz(b) - 1);  ↑9094f2
  int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
  vector<C> L(n), R(n), outs(n), outl(n);
  rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
  rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
  fft(L), fft(R);  ↑f8a1f3
  rep(i,0,n) {
    int j = -i & (n - 1);
    outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
    outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
  }  ↑455f55
  fft(outl), fft(outs);
  rep(i,0,sz(res)) {
    ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
    ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
    res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
  }
  return res;
}  ↑b82773
```

### NumberTheoreticTransform.h

**Description:** ntt(a) computes $\hat{f}(k) = \sum_x a[x] g^{xk}$ for all $k$, where $g = \text{root}^{(mod-1)/N}$. $N$ must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^a b + 1$, where the convolution result has size at most $2^a$. For arbitrary modulo, see FFTMod. `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by $n$, reverse(start+1, end), NTT back. Inputs must be in $[0, \text{mod})$.
**Time:** $\mathcal{O}(N \log N)$

"../number-theory/ModPow.h" d54056, 35 lines

```
const ll mod = (119 << 23) + 1, root = 62; // = 998244353
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). The last two are > 10^9.
typedef vector<ll> vl;  ↑7458ca
void ntt(vl &a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vl rt(2, 1);
  for (static int k = 2, s = 2; k < n; k *= 2, s++) {
    rt.resize(n);
    ll z[] = {1, modpow(root, mod >> s)};  ↑1759b1
    rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod;
  }
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);  ↑158770
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
      ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
      a[i + j + k] = ai - z + (z > ai ? mod : 0);
      ai += (ai + z >= mod ? z - mod : z);
    }  ↑35d5bf
}
vl conv(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};  ↑4001b0
  int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s),
      n = 1 << B;
  ll inv = modpow(n, mod - 2);
  vl L(a), R(b), out(n);
  L.resize(n), R.resize(n);
  ntt(L), ntt(R);  ↑97b4d1
  rep(i,0,n)
    out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
  ntt(out);
  return {out.begin(), out.begin() + s};
}  ↑d54056
```

### FastSubsetTransform.h

**Description:** Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of $a$ must be a power of two.
**Time:** $\mathcal{O}(N \log N)$

464cf3, 16 lines

```
void FST(vi& a, bool inv) {
  for (int n = sz(a), step = 1; step < n; step *= 2) {
    for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step) {
      int &u = a[j], &v = a[j + step]; tie(u, v) =
        inv ? pii(v - u, u) : pii(v, u + v); // AND
        inv ? pii(v, u - v) : pii(u + v, u); // OR
        pii(u + v, u - v);                   // XOR
    }
  }  ↑3431d0
  if (inv) for (int& x : a) x /= sz(a); // XOR only
}
vi conv(vi a, vi b) {
  FST(a, 0); FST(b, 0);
  rep(i,0,sz(a)) a[i] *= b[i];
  FST(a, 1); return a;
}  ↑464cf3
```

# Number theory (5)

## 5.1 Modular arithmetic

### ModInverse.h

**Description:** Pre-computation of modular inverses. Assumes LIM $\le$ mod and that mod is a prime.

6f684f, 3 lines

```
const ll mod = 1000000007, LIM = 200000;
ll* inv = new ll[LIM] - 1; inv[1] = 1;
rep(i,2,LIM) inv[i] = mod - (mod / i) * inv[mod % i] % mod;
```

### ModLog.h

**Description:** Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or $-1$ if no such $x$ exists. modLog(a,1,m) can be used to calculate the order of $a$.
**Time:** $\mathcal{O}\left(\sqrt{m}\right)$

c040b8, 11 lines

```
ll modLog(ll a, ll b, ll m) {
  ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
  unordered_map<ll, ll> A;
  while (j <= n && (e = f = e * a % m) != b % m)
    A[e * b % m] = j++;  ↑2605ad
  if (e == b % m) return j;
  if (__gcd(m, e) == __gcd(m, b))
    rep(i,2,n+2) if (A.count(e = e * f % m))
      return n * i - A[e];
  return -1;
}  ↑c040b8
```

### ModSum.h

**Description:** Sums of mod'ed arithmetic progressions.
modsum(to, c, k, m) $= \sum_{i=0}^{to-1} (ki + c)\%m$. divsum is similar but for floored division.
**Time:** $\log(m)$, with a large constant.

5c5bc5, 16 lines

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }

ull divsum(ull to, ull c, ull k, ull m) {
  ull res = k / m * sumsq(to) + c / m * to;  ↑df3a05
  k %= m; c %= m;
  if (!k) return res;
  ull to2 = (to * k + c) / m;
  return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}  ↑4a574e

ll modsum(ull to, ll c, ll k, ll m) {
  c = ((c % m) + m) % m;
  k = ((k % m) + m) % m;
  return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}  ↑5c5bc5
```

### ModMulLL.h

**Description:** Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \le a, b \le c \le 7.2 \cdot 10^{18}$.
**Time:** $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow

bbbd8f, 11 lines

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
  ll ret = a * b - M * ull(1.L / M * a * b);
  return ret + M * (ret < 0) - M * (ret >= (ll)M);
}  ↑a9c350
ull modpow(ull b, ull e, ull mod) {
  ull ans = 1;
  for (; e; b = modmul(b, b, mod), e /= 2)
    if (e & 1) ans = modmul(ans, b, mod);
  return ans;
}  ↑bbbd8f
```

### ModSqrt.h

**Description:** Tonelli-Shanks algorithm for modular square roots. Finds $x$ s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).
**Time:** $\mathcal{O}\left(\log^2 p\right)$ worst case, $\mathcal{O}(\log p)$ for most $p$

"ModPow.h" 19a793, 24 lines

```
ll sqrt(ll a, ll p) {
  a %= p; if (a < 0) a += p;
  if (a == 0) return 0;
  assert(modpow(a, (p-1)/2, p) == 1); // else no solution
  if (p % 4 == 3) return modpow(a, (p+1)/4, p);  ↑473bd2
  // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
  ll s = p - 1, n = 2;
  int r = 0, m;
  while (s % 2 == 0)
    ++r, s /= 2;  ↑9685a0
  while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
  ll x = modpow(a, (s + 1) / 2, p);
  ll b = modpow(a, s, p), g = modpow(n, s, p);
  for (;; r = m) {
    ll t = b;  ↑deae98
    for (m = 0; m < r && t != 1; ++m)
      t = t * t % p;
    if (m == 0) return x;
    ll gs = modpow(g, 1LL << (r - m - 1), p);
    g = gs * gs % p;  ↑4c1d83
    x = x * gs % p;
    b = b * g % p;
  }
}  ↑19a793
```

## 5.2 Primality

### FastEratosthenes.h

**Description:** Prime sieve for generating all primes smaller than LIM.
**Time:** For LIM $= 10^9$: $\approx 1.5$ s

6b2912, 20 lines

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
  const int S = (int)round(sqrt(LIM)), R = LIM / 2;  ↑a11a60
  vi pr = {2}, sieve(S+1); pr.reserve(int(LIM/log(LIM)*1.1));
  vector<pii> cp;
  for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
    cp.push_back({i, i * i / 2});
    for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
  }  ↑e31824
  for (int L = 1; L <= R; L += S) {
    array<bool, S> block{};
    for (auto &[p, idx] : cp)
      for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
    rep(i,0,min(S, R - L))
      if (!block[i]) pr.push_back((L + i) * 2 + 1);
  }  ↑4de4a4
  for (int i : pr) isPrime[i] = 1;
  return pr;
}  ↑6b2912
```

### MillerRabin.h

**Description:** Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
**Time:** 7 times the complexity of $a^b \mod c$.

"ModMulLL.h" 60dcd1, 12 lines

```
bool isPrime(ull n) {
  if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
  ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
      s = __builtin_ctzll(n-1), d = n >> s;  ↑13b9b1
  for (ull a : A) {   // ^ count trailing zeroes
    ull p = modpow(a%n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
      p = modmul(p, p, n);
    if (p != n-1 && i != s) return 0;
  }
  return 1;
}  ↑60dcd1
```

### Factor.h

**Description:** Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 → {11, 19, 11}).
**Time:** $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMulLL.h", "MillerRabin.h" d8d98d, 18 lines

```
ull pollard(ull n) {
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ull x) { return modmul(x, x, n) + i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);  ↑47de4d
    if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
    x = f(x), y = f(f(y));
  }
  return __gcd(prd, n);
}  ↑cd2ac3
vector<ull> factor(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);  ↑0d9093
  l.insert(l.end(), all(r));
  return l;
}  ↑d8d98d
```

## 5.3 Divisibility

### Euclid.h

**Description:** Finds two integers $x$ and $y$, such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in __gcd instead. If $a$ and $b$ are coprime, then $x$ is the inverse of $a \pmod b$.

33ba8f, 5 lines

```
ll euclid(ll a, ll b, ll &x, ll &y) {
  if (!b) return x = 1, y = 0, a;
  ll d = euclid(b, a % b, y, x);
  return y -= a/b * x, d;
}  ↑33ba8f
```

### CRT.h

**Description:** Chinese Remainder Theorem.
crt(a, m, b, n) computes $x$ such that $x \equiv a \pmod m$, $x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, $x$ will obey $0 \le x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
**Time:** $\log(n)$

"Euclid.h" 04d93a, 7 lines

```
ll crt(ll a, ll m, ll b, ll n) {
  if (n > m) swap(a, b), swap(m, n);
  ll x, y, g = euclid(m, n, x, y);
  assert((a - b) % g == 0); // else no solution
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m*n/g : x;
}  ↑04d93a
```

#### 5.3.1 Bézout's identity

For $a \neq, b \neq 0$, then $d = gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If $(x, y)$ is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

### PhiFunction.h

**Description:** *Euler's* $\phi$ function is defined as $\phi(n) :=$ # of positive integers $\le n$ that are coprime with $n$. $\phi(1) = 1$, $p$ prime $\Rightarrow \phi(p^k) = (p-1)p^{k-1}$, $m, n$ coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1} p_2^{k_2} ... p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1} ... (p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n}(1 - 1/p)$.
$\sum_{d|n} \phi(d) = n$, $\sum_{1 \le k \le n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$
**Euler's thm**: $a, n$ coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.

**Fermat's little thm**: $p$ prime $\Rightarrow a^{p-1} \equiv 1 \pmod p$ $\forall a$.

cf7d6d, 8 lines

```
const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
  rep(i,0,LIM) phi[i] = i&1 ? i : i/2;  ↑70ba16
  for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
    for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}  ↑cf7d6d
```

## 5.4 Fractions

### ContinuedFractions.h

**Description:** Given $N$ and a real number $x \geq 0$, finds the closest rational approximation $p/q$ with $p, q \leq N$. It will obey $|p/q - x| \leq 1/qN$.

For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. ($p_k/q_k$ alternates between $> x$ and $< x$.) If $x$ is rational, $y$ eventually becomes $\infty$; if $x$ is the root of a degree 2 polynomial the $a$'s eventually become cyclic.

**Time:** $\mathcal{O}(\log N)$

dd6c5e, 21 lines

```
typedef double d; // for N ~ 1e7; long double for N ~ 1e9
pair<ll, ll> approximate(d x, ll N) {
  ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x;
  for (;;) {
    ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf),
       a = (ll)floor(y), b = min(a, lim),
       NP = b*P + LP, NQ = b*Q + LQ;  ↑12b990
    if (a > b) {
      // If b > a/2, we have a semi-convergent that gives us a
      // better approximation; if b = a/2, we *may* have one.
      // Return {P, Q} here for a more canonical approximation.
      return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)) ?
        make_pair(NP, NQ) : make_pair(P, Q);
    }  ↑451a2f
    if (abs(y = 1/(y - (d)a)) > 3*N) {
      return {NP, NQ};
    }
    LP = P; P = NP;
    LQ = Q; Q = NQ;
  }
}  ↑dd6c5e
```

### FracBinarySearch.h

**Description:** Given $f$ and $N$, finds the smallest fraction $p/q \in [0,1]$ such that $f(p/q)$ is true, and $p, q \leq N$. You may want to throw an exception from $f$ if it finds an exact solution, in which case $N$ can be removed.

**Usage:** `fracBS([](Frac f) { return f.p>=3*f.q; }, 10); // {1,3}`

**Time:** $\mathcal{O}(\log(N))$

27ab3e, 25 lines

```
struct Frac { ll p, q; };

template<class F>
Frac fracBS(F f, ll N) {
  bool dir = 1, A = 1, B = 1;  ↑38638e
  Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
  if (f(lo)) return lo;
  assert(f(hi));  ↑eb728d
  while (A || B) {
    ll adv = 0, step = 1; // move hi if dir, else lo
    for (int si = 0; step; (step *= 2) >>= si) {
      adv += step;
      Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
      if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
        adv -= step; si = 2;
      }  ↑a40ec9
    }
    hi.p += lo.p * adv;
    hi.q += lo.q * adv;
    dir = !dir;  ↑758409
    swap(lo, hi);
    A = B; B = !!adv;
  }
  return dir ? hi : lo;
}  ↑27ab3e
```

## 5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2),\ \ b = k \cdot (2mn),\ \ c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either $m$ or $n$ even.

## 5.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use $7 \cdot 10^{18} + 6799$. There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power $p^a$, except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

## 5.7 Estimates

$\sum_{d|n} d = O(n \log \log n)$.

The number of divisors of $n$ is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

## 5.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g(n/d)$$

Other useful formulas/forms:

$\sum_{d|n} \mu(d) = [n = 1]$ (very useful)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n) g(d)$

$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$

# Combinatorial (6)

## 6.1 Permutations

### 6.1.1 Factorial

| $n$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| $n!$ | 1 | 2 | 6 | 24 | 120 | 720 | 5040 | 40320 | 362880 | 3628800 |
| $n$ | 11 | 12 | 13 | 14 | 15 | 16 | 17 | | | |
| $n!$ | 4.0e7 | 4.8e8 | 6.2e9 | 8.7e10 | 1.3e12 | 2.1e13 | 3.6e14 | | | |
| $n$ | 20 | 25 | 30 | 40 | 50 | 100 | 150 | 171 | | |
| $n!$ | 2e18 | 2e25 | 3e32 | 8e47 | 3e64 | 9e157 | 6e262 | > DBL_MAX | | |

### IntPerm.h

**Description:** Permutation $\rightarrow$ integer conversion. (Not order preserving.) Integer $\rightarrow$ permutation can use a lookup table.

**Time:** $\mathcal{O}(n)$

044568, 6 lines

```
int permToInt(vi& v) {
  int use = 0, i = 0, r = 0;
  for(int x:v) r = r * ++i + __builtin_popcount(use & -(1<<x)),
    use |= 1 << x;                    // (note: minus, not ~!)
  return r;
}  ↑044568
```

### 6.1.2 Cycles

Let $g_S(n)$ be the number of $n$-permutations whose cycle lengths all belong to the set $S$. Then

$$\sum_{n=0}^{\infty} g_S(n) \frac{x^n}{n!} = \exp\left(\sum_{n \in S} \frac{x^n}{n}\right)$$

### 6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

### 6.1.4 Burnside's lemma

Given a group $G$ of symmetries and a set $X$, the number of elements of $X$ *up to symmetry* equals

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

where $X^g$ are the elements fixed by $g$ ($g.x = x$).

If $f(n)$ counts "configurations" (of some sort) of length $n$, we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n} \sum_{k|n} f(k) \phi(n/k).$$

## 6.2 Partitions and subsets

### 6.2.1 Partition function

Number of ways of writing $n$ as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1,\ p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

| $n$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 20 | 50 | 100 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $p(n)$ | 1 | 1 | 2 | 3 | 5 | 7 | 11 | 15 | 22 | 30 | 627 | $\sim 2 \cdot 10^5$ | $\sim 2 \cdot 10^8$ |

### 6.2.2 Lucas' Theorem

Let $n, m$ be non-negative integers and $p$ a prime. Write $n = n_k p^k + \ldots + n_1 p + n_0$ and $m = m_k p^k + \ldots + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^{k} \binom{n_i}{m_i} \pmod p$.

### 6.2.3 Binomials

Multinomial.h

**Description:** Computes $\binom{k_1+\cdots+k_n}{k_1,k_2,\ldots,k_n} = \frac{(\sum k_i)!}{k_1!k_2!\ldots k_n!}$. a0a312, 5 lines

```
ll multinomial(vi& v) {
  ll c = 1, m = v.empty() ? 1 : v[0];
  rep(i,1,sz(v)) rep(j,0,v[i]) c = c * ++m / (j+1);
  return c;
} ↑a0a312
```

## 6.3 General purpose numbers

### 6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t-1}$ (FFT-able).
$B[0,\ldots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \ldots]$

Sums of powers:

$$\sum_{i=1}^{n} n^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k}B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x)dx - \sum_{k=1}^{\infty}\frac{B_k}{k!}f^{(k-1)}(m)$$

$$\approx \int_m^{\infty} f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

### 6.3.2 Stirling numbers of the first kind

Number of permutations on $n$ items with $k$ cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$

$$\textstyle\sum_{k=0}^{n} c(n,k)x^k = x(x+1)\ldots(x+n-1)$$

$c(8,k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \ldots$

### 6.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly $k$ elements are greater than the previous element. $k$ $j$:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ $j$:s s.t. $\pi(j) \geq j$, $k$ $j$:s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$

$$E(n,0) = E(n,n-1) = 1$$

$$E(n,k) = \sum_{j=0}^{k}(-1)^j\binom{n+1}{j}(k+1-j)^n$$

### 6.3.4 Stirling numbers of the second kind

Partitions of $n$ distinct elements into exactly $k$ groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$

$$S(n,1) = S(n,n) = 1$$

$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$$

### 6.3.5 Bell numbers

Total number of partitions of $n$ distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \ldots.$ For $p$ prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod{p}$$

### 6.3.6 Labeled unrooted trees

# on $n$ vertices: $n^{n-2}$
# on $k$ existing trees of size $n_i$: $n_1 n_2 \cdots n_k n^{k-2}$
# with degrees $d_i$: $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

### 6.3.7 Catalan numbers

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \ldots$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with $n$ pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

# Graph (7)

## 7.1 Network flow

PushRelabel.h

**Description:** Push-relabel using the highest label selection rule and the gap heuristic. Quite fast in practice. To obtain the actual flow, look at positive values only.

**Time:** $\mathcal{O}\left(V^2\sqrt{E}\right)$ 0ae1d4, 48 lines

```
struct PushRelabel {
  struct Edge {
    int dest, back;
    ll f, c;
  }; ↑d82272
  vector<vector<Edge>> g;
  vector<ll> ec;
  vector<Edge*> cur;
  vector<vi> hs; vi H; ↑786b95
  PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}

  void addEdge(int s, int t, ll cap, ll rcap=0) {
    if (s == t) return;
    g[s].push_back({t, sz(g[t]), 0, cap});
    g[t].push_back({s, sz(g[s])-1, 0, rcap});
  } ↑c1743e

  void addFlow(Edge& e, ll f) {
    Edge &back = g[e.dest][e.back]; ↑e5e9ea
    if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
    e.f += f; e.c -= f; ec[e.dest] += f;
    back.f -= f; back.c += f; ec[back.dest] -= f;
  } ↑b328ff
  ll calc(int s, int t) {
    int v = sz(g); H[s] = v; ec[t] = 1;
    vi co(2*v); co[0] = v-1;
    rep(i,0,v) cur[i] = g[i].data();
    for (Edge& e : g[s]) addFlow(e, e.c); ↑e82137

    for (int hi = 0;;) {
      while (hs[hi].empty()) if (!hi--) return -ec[s];
      int u = hs[hi].back(); hs[hi].pop_back();
      while (ec[u] > 0)  // discharge u
        if (cur[u] == g[u].data() + sz(g[u])) {
          H[u] = 1e9; ↑57b5e9
          for (Edge& e : g[u]) if (e.c && H[u] > H[e.dest]+1)
            H[u] = H[e.dest]+1, cur[u] = &e;
          if (++co[H[u]], !--co[hi] && hi < v)
            rep(i,0,v) if (hi < H[i] && H[i] < v)
              --co[H[i]], H[i] = v + 1; ↑d60ed0
          hi = H[u];
        } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)
          addFlow(*cur[u], min(ec[u], cur[u]->c));
        else ++cur[u];
    } ↑10d806
  }
  bool leftOfMinCut(int a) { return H[a] >= sz(g); }
}; ↑0ae1d4
```

MinCostMaxFlow.h

**Description:** Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.
**Time:** $\mathcal{O}(FE\log(V))$ where $F$ is max flow. $\mathcal{O}(VE)$ for setpi. 71ef77, 79 lines

```
#include <bits/extc++.h>

const ll INF = numeric_limits<ll>::max() / 4; ↑fe49b8

struct MCMF {
  struct edge {
    int from, to, rev;
    ll cap, cost, flow;
  }; ↑f6428b
  int N;
  vector<vector<edge>> ed;
  vi seen;
  vector<ll> dist, pi;
  vector<edge*> par; ↑c0aa35

  MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

  void addEdge(int from, int to, ll cap, ll cost) {
    if (from == to) return;
    ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost,0 });
    ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost,0 });
  } ↑76dfc2

  void path(int s) {
    fill(all(seen), 0); ↑b25d5d
    fill(all(dist), INF);
    dist[s] = 0; ll di;

    __gnu_pbds::priority_queue<pair<ll, int>> q;
    vector<decltype(q)::point_iterator> its(N); ↑9e327a
    q.push({ 0, s });

    while (!q.empty()) {
      s = q.top().s; q.pop();
      seen[s] = 1; di = dist[s] + pi[s]; ↑bd8061
      for (edge& e : ed[s]) if (!seen[e.to]) {
```

```
      ll val = di - pi[e.to] + e.cost;
      if (e.cap - e.flow > 0 && val < dist[e.to]) {
        dist[e.to] = val;
        par[e.to] = &e; ↑371535
        if (its[e.to] == q.end())
          its[e.to] = q.push({ -dist[e.to], e.to });
        else
          q.modify(its[e.to], { -dist[e.to], e.to });
      } ↑4c1a92
    }
  }
  rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
} ↑8fc539

pair<ll, ll> maxflow(int s, int t) {
  ll totflow = 0, totcost = 0;
  while (path(s), seen[t]) {
    ll fl = INF; ↑fd6afe
    for (edge* x = par[t]; x; x = par[x->from])
      fl = min(fl, x->cap - x->flow);

    totflow += fl;
    for (edge* x = par[t]; x; x = par[x->from]) {
      x->flow += fl; ↑8e0eca
      ed[x->to][x->rev].flow -= fl;
    }
  }
  rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e.flow;
  return {totflow, totcost/2};
} ↑7b5c00

// If some costs can be negative, call this before maxflow:
void setpi(int s) { // (otherwise, leave this out)
  fill(all(pi), INF); pi[s] = 0; ↑72533c
  int it = N, ch = 1; ll v;
  while (ch-- && it--)
    rep(i,0,N) if (pi[i] != INF)
      for (edge& e : ed[i]) if (e.cap)
        if ((v = pi[i] + e.cost) < pi[e.to])
          pi[e.to] = v, ch = 1; ↑520651
  assert(it >= 0); // negative cost cycle
}
}; ↑71ef77
```

## Dinic.h

**Description:** Flow algorithm with complexity $O(VE\log U)$ where $U = \max|\text{cap}|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)$ for bipartite matching.

d7f0f1, 42 lines

```
struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, 0LL); } // if you need flows
  }; ↑9d5927
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {} ↑fdd5b9
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    adj[a].push_back({b, sz(adj[b]), c, c});
    adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
  } ↑a45d7e
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < sz(adj[v]); i++) {
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p; ↑aedea1
          return p;
        }
    }
    return 0;
  } ↑72048c
  ll calc(int s, int t) {
    ll flow = 0; q[0] = s;
    rep(L,0,31) do { // 'int L=30' maybe faster for random data
      lvl = ptr = vi(sz(q));
      int qi = 0, qe = lvl[s] = 1; ↑5d9371
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (Edge e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      } ↑16dd6b
      while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
    } while (lvl[t]);
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0; }
}; ↑d7f0f1
```

## MinCut.h

**Description:** After running max-flow, the left side of a min-cut from $s$ to $t$ is given by all vertices reachable from $s$, only traversing edges with positive residual capacity.

## GlobalMinCut.h

**Description:** Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.
**Time:** $\mathcal{O}(V^3)$

8b0e19, 21 lines

```
pair<int, vi> globalMinCut(vector<vi> mat) {
  pair<int, vi> best = {INT_MAX, {}};
  int n = sz(mat);
  vector<vi> co(n);
  rep(i,0,n) co[i] = {i}; ↑f640ab
  rep(ph,1,n) {
    vi w = mat[0];
    size_t s = 0, t = 0;
    rep(it,0,n-ph) { // O(V^2) -> O(E log V) with prio. queue
      w[t] = INT_MIN; ↑c98135
      s = t, t = max_element(all(w)) - w.begin();
      rep(i,0,n) w[i] += mat[t][i];
    } ↑8c07c9
    best = min(best, {w[t] - mat[t][t], co[t]});
    co[s].insert(co[s].end(), all(co[t]));
    rep(i,0,n) mat[s][i] += mat[t][i];
    rep(i,0,n) mat[i][s] = mat[s][i];
    mat[0][t] = INT_MIN;
  }
  return best;
} ↑8b0e19
```

## GomoryHu.h

**Description:** Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.
**Time:** $\mathcal{O}(V)$ Flow Computations

"PushRelabel.h" 0418b3, 13 lines

```
typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
  vector<Edge> tree;
  vi par(N); ↑2ac223
  rep(i,1,N) {
    PushRelabel D(N); // Dinic also works
    for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
    tree.push_back({i, par[i], D.calc(i, par[i])}); ↑53151e
    rep(j,i+1,N)
      if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
  }
  return tree;
} ↑0418b3
```

# 7.2 Matching

## HopcroftKarp.h

**Description:** Fast bipartite matching algorithm. Graph $g$ should be a list of neighbors of the left partition, and $r$ should be a vector full of $-1$'s of the same size as the right partition. Returns the size of the matching. $r[i]$ will be the match for vertex $i$ on the right side, or $-1$ if it's not matched.
**Time:** $\mathcal{O}\left(E\sqrt{V}\right)$

731cfb, 20 lines

```
int hopcroftKarp(vector<vi>& g, vi& r) {
  int n = sz(g), res = 0;
  vi l(n, -1), q(n), d(n);
  auto dfs = [&](auto f, int u) -> bool {
    int t = exchange(d[u], 0) + 1; ↑127bc2
    for (int v : g[u])
      if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
        return l[u] = v, r[v] = u, 1;
    return 0;
  }; ↑f7ade4
  for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
    rep(i,0,n) if (l[i] == -1) q[t++] = i, d[i] = 1;
    rep(i,0,t) for (int v : g[q[i]]) {
      if (r[v] == -1) f = 1;
      else if (!d[r[v]]) d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
    } ↑7149a0
    if (!f) return res;
    rep(i,0,n) if (l[i] == -1) res += dfs(dfs, i);
  }
} ↑731cfb
```

## MinimumVertexCover.h

**Description:** Finds a minimum vertex cover in a bipartite graph. $g$ lists neighbors of the left partition. Requires an existing maximum matching $r$ from HopcroftKarp, where $r[v]$ is the left match of right vertex $v$, or $-1$. Returns two 0/1 vectors $cl, cr$, indexed within each side: $cl[u] = 1$ iff left vertex $u$ is in the cover, and $cr[v] = 1$ iff right vertex $v$ is in the cover. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.
**Usage:** `vi r(m, -1); hopcroftKarp(g, r);`
`auto [cl, cr] = cover(g, r);`
**Time:** $\mathcal{O}(V + E)$, given a maximum matching.

"HopcroftKarp.h" 70f30c, 11 lines

```
pair<vi, vi> cover(vector<vi>& g, vi& r) {
  int n = sz(g), t = 0;
  vi cl(n), cr(sz(r)), q(n);
  for (int u : r) if (u != -1) cl[u] = 1;
  rep(i,0,n) if (!cl[i]) q[t++] = i; ↑cd1890
  rep(i,0,t) for (int v : g[q[i]]) {
    cr[v] = 1;
    if (r[v] != -1 && cl[r[v]]) cl[q[t++] = r[v]] = 0;
  }
  return {cl, cr};
} ↑70f30c
```

## Hungarian.h

**Description:** Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes cost$[N][M]$, where cost$[i][j]$ = cost for $L[i]$ to be matched with $R[j]$, and returns (min cost, match), where $L[i]$ is matched with $R[\text{match}[i]]$. Negate costs for max cost. Requires $N \le M$.

**Time:** $\mathcal{O}\left(N^2M\right)$

66a0f9, 24 lines

```
template<class T>
pair<T, vi> hungarian(vector<vector<T>> &C) {
  int i = sz(C), m = i ? sz(C[0]) : 0, c, s, r;
  vector<T> dist(m), pot(m);
  vi match(i), rev(m, -1), cols(m), prev(m); ↑de4948
  T d = 0, nd, cost = 0;
  while (i--) {
    rep(c,0,m) dist[c] = C[i][c], cols[c] = c, prev[c] = i;
    for (s = 0;;) {
      rep(j,s,m) {
        c = cols[j], nd = dist[c] - pot[c]; ↑189245
        if (j == s || d > nd) d = nd, swap(cols[s], cols[j]);
      }
      if ((r = rev[c = cols[s++]]) == -1) break;
      rep(j,0,m) if (dist[j] > (nd = C[r][j]-C[r][c]+dist[c]))
        dist[j] = nd, prev[j] = r;
    } ↑991b3e
    cost += dist[c];
    while (s--) pot[cols[s]] = dist[cols[s]] - d;
    for (; r != i; swap(c, match[r]))
      r = rev[c] = prev[c]; ↑634d76
  }
  return {cost, match};
} ↑66a0f9
```

## GeneralMatching.h

**Description:** Matching for general graphs. Fails with probability $N/mod$.
**Time:** $\mathcal{O}\left(N^3\right)$

"../numerical/MatrixInverse-mod.h" 139ef7, 40 lines

```
vector<pii> generalMatching(int N, vector<pii>& ed) {
  vector<vector<ll>> mat(N, vector<ll>(N)), A;
  for (pii pa : ed) {
    int a = pa.f, b = pa.s, r = rand() % mod;
    mat[a][b] = r, mat[b][a] = (mod - r) % mod;
  } ↑c0eecf

  int r = matInv(A = mat), M = 2*N - r, fi, fj;
  assert(r % 2 == 0); ↑39c1d8

  if (M != N) do {
    mat.resize(M, vector<ll>(M));
    rep(i,0,N) {
      mat[i].resize(M); ↑775ad0
      rep(j,N,M) {
        int r = rand() % mod;
        mat[i][j] = r, mat[j][i] = (mod - r) % mod;
      }
    } ↑b88c5b
  } while (matInv(A = mat) != M);

  vi has(M, 1); vector<pii> ret;
  rep(it,0,M/2) {
    rep(i,0,M) if (has[i])
      rep(j,i+1,M) if (A[i][j] && mat[i][j]) {
        fi = i; fj = j; goto done; ↑e3e8ce
    } assert(0); done:
    if (fj < N) ret.emplace_back(fi, fj);
    has[fi] = has[fj] = 0; ↑c9f31c
    rep(sw,0,2) {
      ll a = modpow(A[fi][fj], mod-2);
      rep(i,0,M) if (has[i] && A[i][fj]) {
        ll b = A[i][fj] * a % mod;
        rep(j,0,M) A[i][j] = (A[i][j] - A[fi][j] * b) % mod;
      } ↑3c2fa2
      swap(fi,fj);
    }
  }
  return ret;
} ↑139ef7
```

## MatroidIntersection.h

**Description:** Returns a largest common independent set of two matroids on elements $[0, n)$, in sorted order. Each oracle must provide clear() (reset to the empty set), add(x) (insert an element known to preserve independence), and check(x) (test whether x can be added independently, without changing the represented set). The oracles must have separate state; their initial contents are ignored, and both contain the returned set on exit.
**Time:** $\mathcal{O}\left(n(r+1)^2T\right)$, where $r$ is the answer size and $T$ bounds each oracle operation. Uses $\mathcal{O}\left(n(r+1)\right)$ extra space.

0eb105, 40 lines

```
template<class M1, class M2>
vi matroidIntersection(M1& m1, M2& m2, int n) {
  vi in(n);
  for (;;) {
    vi cur, out; ↑97d0ef
    rep(x,0,n) (in[x] ? cur : out).push_back(x);
    auto build = [&](int skip) {
      m1.clear(); m2.clear();
      for (int x : cur) if (x != skip)
        m1.add(x), m2.add(x);
    }; ↑aba44c
    int s = n, t = n + 1;
    vector<vi> adj(n + 2);
    build(-1); ↑7b4fe0
    for (int x : out) {
      if (m1.check(x)) adj[s].push_back(x);
      if (m2.check(x)) adj[x].push_back(t);
    } ↑fc8ec8
    for (int y : cur) {
      build(y);
      for (int x : out) {
        if (m1.check(x)) adj[y].push_back(x);
        if (m2.check(x)) adj[x].push_back(y);
      }
    } ↑e3fda8
    vi par(n + 2, -1), q = {s};
    par[s] = s;
    rep(i,0,sz(q)) {
      if (par[t] != -1) break;
      for (int v : adj[q[i]]) if (par[v] == -1)
        par[v] = q[i], q.push_back(v);
    } ↑11c6cc
    if (par[t] == -1) {
      build(-1);
      return cur;
    } ↑1a112c
    for (int x = par[t]; x != s; x = par[x])
      in[x] ^= 1;
  }
} ↑0eb105
```

# 7.3 DFS algorithms

## SCC.h

**Description:** Finds strongly connected components in a directed graph. If vertices $u, v$ belong to the same component, we can reach $u$ from $v$ and vice versa.
**Usage:** `scc(graph, [&](vi& v) { ... })` visits all components in reverse topological order. `comp[i]` holds the component index of a node (a component only has edges to components with lower index). `ncomps` will contain the number of components.
**Time:** $\mathcal{O}\left(E+V\right)$

76b5c9, 24 lines

```
vi val, comp, z, cont;
int Time, ncomps;
template<class G, class F> int dfs(int j, G& g, F& f) {
  int low = val[j] = ++Time, x; z.push_back(j); ↑9b6eaf
  for (auto e : g[j]) if (comp[e] < 0)
    low = min(low, val[e] ?: dfs(e,g,f));

  if (low == val[j]) {
    do {
      x = z.back(); z.pop_back(); ↑b81dc2
      comp[x] = ncomps;
      cont.push_back(x);
    } while (x != j);
    f(cont); cont.clear();
    ncomps++;
  } ↑6e1ce2
  return val[j] = low;
}
template<class G, class F> void scc(G& g, F f) {
  int n = sz(g); ↑8d248c
  val.assign(n, 0); comp.assign(n, -1);
  Time = ncomps = 0;
  rep(i,0,n) if (comp[i] < 0) dfs(i, g, f);
} ↑76b5c9
```

## Dominator.h

**Description:** Computes the dominator tree of a directed graph. A vertex u dominates v if every path from root to v passes through u. Vertices are numbered from 0 to n-1. After calc, ans contains the dominator tree's children; parent(v) is v's immediate dominator, or -1 for the root and unreachable vertices. Add all edges before calling calc, and call calc exactly once with a valid root.
**Usage:** `Dominator d(n); d.ae(a, b); d.calc(root);`
**Time:** $\mathcal{O}\left((V+E)\log V\right)$

c129b4, 47 lines

```
struct Dominator {
  int co = 0;
  vector<vi> adj, ans, radj, child, bucket;
  vi label, rlabel, sdom, dom, par, best; ↑5807cf
  Dominator(int n) : adj(n), ans(n), radj(n+1),
    child(n+1), bucket(n+1), label(n), rlabel(n+1, -1),
    sdom(n+1), dom(n+1), par(n+1), best(n+1) {}
  void ae(int a, int b) { adj[a].push_back(b); }
  int get(int x) {
    if (par[x] != x) {
      int t = get(par[x]); ↑6a0189
      par[x] = par[par[x]];
      if (sdom[t] < sdom[best[x]]) best[x] = t;
    } ↑805077
    return best[x];
  }
  void dfs(int x) {
    label[x] = ++co;
    rlabel[co] = x;
    sdom[co] = par[co] = best[co] = co; ↑29ce7a
    for (int y : adj[x]) {
      if (!label[y]) {
        dfs(y);
        child[label[x]].push_back(label[y]);
      } ↑5ea4ae
      radj[label[y]].push_back(label[x]);
    }
  } ↑a31928
  void calc(int root) {
    dfs(root);
    for (int i = co; i >= 1; --i) {
      for (int j : radj[i])
        sdom[i] = min(sdom[i], sdom[get(j)]);
      if (i > 1) bucket[sdom[i]].push_back(i); ↑c560a3
      for (int j : bucket[i]) {
        int k = get(j);
        dom[j] = sdom[j] == sdom[k] ? sdom[j] : k;
      }
```

```
    for (int j : child[i]) par[j] = i;
  } ↑598d99
  rep(i,2,co+1) {
    if (dom[i] != sdom[i]) dom[i] = dom[dom[i]];
    ans[rlabel[dom[i]]].push_back(rlabel[i]);
  }
 }
 int parent(int x) { return rlabel[dom[label[x]]]; }
}; ↑c129b4
```

## BiconnectedComponents.h

**Description:** Finds all biconnected components in an undirected graph, and runs a callback for the edges in each. In a biconnected component there are at least two internally disjoint paths between any two nodes (a cycle exists through them). Note that a node can be in several components. An edge which is not in a component is a bridge, i.e., not part of any cycle.

**Usage:** `int eid = 0; ed.resize(N);`
`for each edge (a,b) {`
`ed[a].emplace_back(b, eid);`
`ed[b].emplace_back(a, eid++); }`
`bicomps([&](const vi& edgelist) {...});`

**Time:** $\mathcal{O}(E+V)$

c6b7c7, 32 lines

```
vi num, st;
vector<vector<pii>> ed;
int Time;
template<class F>
int dfs(int at, int par, F& f) {
 int me = num[at] = ++Time, top = me; ↑084daa
 for (auto [y, e] : ed[at]) if (e != par) {
  if (num[y]) {
   top = min(top, num[y]); ↑988c81
   if (num[y] < me)
    st.push_back(e);
  } else {
   int si = sz(st);
   int up = dfs(y, e, f);
   top = min(top, up); ↑21f1da
   if (up == me) {
    st.push_back(e);
    f(vi(st.begin() + si, st.end()));
    st.resize(si);
   } ↑35c5c9
   else if (up < me) st.push_back(e);
   else { /* e is a bridge */ }
  }
 } ↑74637b
 return top;
}

template<class F>
void bicomps(F f) {
 num.assign(sz(ed), 0);
 rep(i,0,sz(ed)) if (!num[i]) dfs(i, -1, f);
} ↑c6b7c7
```

## TwoSat.h

**Description:** Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||!b)\&\&\ldots$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).

**Usage:** `TwoSat ts(number of boolean variables);`
`ts.either(0, ~3); // Var 0 is true or var 3 is false`
`ts.setValue(2); // Var 2 is true`
`ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true`
`ts.solve(); // Returns true iff it is solvable`
`ts.values[0..N-1] holds the assigned values to the vars`

**Time:** $\mathcal{O}(N+E)$, where $N$ is the number of boolean variables, and $E$ is the number of clauses.

5f9706, 56 lines

```
struct TwoSat {
 int N;
 vector<vi> gr;
 vi values; // 0 = false, 1 = true

 TwoSat(int n = 0) : N(n), gr(2*n) {} ↑c1fbac

 int addVar() { // (optional)
  gr.emplace_back();
  gr.emplace_back(); ↑ca34a5
  return N++;
 }

 void either(int f, int j) {
  f = max(2*f, -1-2*f); ↑5e1028
  j = max(2*j, -1-2*j);
  gr[f].push_back(j^1);
  gr[j].push_back(f^1);
 } ↑f602cc
 void setValue(int x) { either(x, x); }

 void atMostOne(const vi& li) { // (optional)
  if (sz(li) <= 1) return;
  int cur = ~li[0]; ↑7721c4
  rep(i,2,sz(li)) {
   int next = addVar();
   either(cur, ~li[i]);
   either(cur, next);
   either(~li[i], next);
   cur = ~next;
  } ↑7cdc2a
  either(cur, ~li[1]);
 }

 vi val, comp, z; int time = 0; ↑594dbb
 int dfs(int i) {
  int low = val[i] = ++time, x; z.push_back(i);
  for(int e : gr[i]) if (!comp[e])
   low = min(low, val[e] ?: dfs(e)); ↑c634a9
  if (low == val[i]) do {
   x = z.back(); z.pop_back();
   comp[x] = low;
   if (values[x>>1] == -1)
    values[x>>1] = x&1;
  } while (x != i); ↑5a0145
  return val[i] = low;
 }

 bool solve() {
  values.assign(N, -1); ↑07c73a
  val.assign(2*N, 0); comp = val;
  rep(i,0,2*N) if (!comp[i]) dfs(i);
  rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
  return 1;
 }
}; ↑5f9706
```

## EulerWalk.h

**Description:** Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .s to s and ret.

**Time:** $\mathcal{O}(V+E)$

780b64, 15 lines

```
vi eulerWalk(vector<vector<pii>>& gr, int nedges, int src=0) {
 int n = sz(gr);
 vi D(n), its(n), eu(nedges), ret, s = {src}; ↑ea6179
 D[src]++; // to allow Euler paths, not just cycles
 while (!s.empty()) {
  int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
  if (it == end){ ret.push_back(x); s.pop_back(); continue; }
  tie(y, e) = gr[x][it++];
  if (!eu[e]) {
   D[x]--, D[y]++; ↑93be2a
   eu[e] = 1; s.push_back(y);
  }}
 for (int x : D) if (x < 0 || sz(ret) != nedges+1) return {};
 return {ret.rbegin(), ret.rend()};
} ↑780b64
```

# 7.4 Coloring

## EdgeColoring.h

**Description:** Given a simple, undirected graph with max degree $D$, computes a $(D+1)$-coloring of the edges such that no neighboring edges share a color. ($D$-coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)

**Time:** $\mathcal{O}(NM)$

9cce12, 31 lines

```
vi edgeColoring(int N, vector<pii> eds) {
 vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
 for (pii e : eds) ++cc[e.f], ++cc[e.s];
 int u, v, ncols = *max_element(all(cc)) + 1;
 vector<vi> adj(N, vi(ncols, -1)); ↑94399d
 for (pii e : eds) {
  tie(u, v) = e;
  fan[0] = v;
  loc.assign(ncols, 0); ↑26e011
  int at = u, end = u, d, c = free[u], ind = 0, i = 0;
  while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
   loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
  cc[loc[d]] = c;
  for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
   swap(adj[at][cd], adj[end = at][cd ^ c ^ d]); ↑d631ee
  while (adj[fan[i]][d] != -1) {
   int left = fan[i], right = fan[++i], e = cc[i];
   adj[u][e] = left;
   adj[left][e] = u;
   adj[right][e] = -1; ↑f144e8
   free[right] = e;
  }
  adj[u][d] = fan[i];
  adj[fan[i]][d] = u;
  for (int y : {fan[0], u, end})
   for (int& z = free[y] = 0; adj[y][z] != -1; z++);
 } ↑c722e1
 rep(i,0,sz(eds))
  for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
 return ret;
} ↑9cce12
```

# 7.5 Heuristics

## MaximalCliques.h

**Description:** Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.

**Time:** $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

b0d5b1, 12 lines

```
typedef bitset<128> B;
template<class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={}) {
 if (!P.any()) { if (!X.any()) f(R); return; }
 auto q = (P | X)._Find_first(); ↑abbe26
 auto cands = P & ~eds[q];
 rep(i,0,sz(eds)) if (cands[i]) {
```

```
    R[i] = 1;
    cliques(eds, f, P & eds[i], X & eds[i], R);
    R[i] = P[i] = 0; X[i] = 1;
  }
} ↑b0d5b1
```

## MaximumClique.h

**Description:** Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.
**Time:** Runs in about 1s for $n = 155$ and worst case random graphs ($p = .90$). Runs faster for sparse graphs.

f7c0bc, 49 lines

```
typedef vector<bitset<200>> vb;
struct Maxclique {
  double limit=0.025, pk=0;
  struct Vertex { int i, d=0; };
  typedef vector<Vertex> vv; ↑b929e8
  vb e;
  vv V;
  vector<vi> C;
  vi qmax, q, S, old; ↑b548bf
  void init(vv& r) {
    for (auto& v : r) v.d = 0;
    for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
    sort(all(r), [](auto a, auto b) { return a.d > b.d; });
    int mxD = r[0].d;
    rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
  } ↑d5dc84
  void expand(vv& R, int lev = 1) {
    S[lev] += S[lev - 1] - old[lev];
    old[lev] = S[lev - 1]; ↑8602ba
    while (sz(R)) {
      if (sz(q) + R.back().d <= sz(qmax)) return;
      q.push_back(R.back().i);
      vv T;
      for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
      if (sz(T)) {
        if (S[lev]++ / ++pk < limit) init(T); ↑c706bf
        int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
        C[1].clear(), C[2].clear();
        for (auto v : T) {
          int k = 1; ↑d6bf0a
          auto f = [&](int i) { return e[v.i][i]; };
          while (any_of(all(C[k]), f)) k++;
          if (k > mxk) mxk = k, C[mxk + 1].clear();
          if (k < mnk) T[j++].i = v.i;
          C[k].push_back(v.i);
        } ↑5ebe7a
        if (j > 0) T[j - 1].d = 0;
        rep(k,mnk,mxk + 1) for (int i : C[k])
          T[j].i = i, T[j++].d = k;
        expand(T, lev + 1); ↑e72ba9
      } else if (sz(q) > sz(qmax)) qmax = q;
      q.pop_back(), R.pop_back();
    }
  } ↑901020
  vi maxClique() { init(V), expand(V); return qmax; }
  Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
    rep(i,0,sz(e)) V.push_back({i});
  }
}; ↑f7c0bc
```

## MaximumIndependentSet.h

**Description:** To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see MinimumVertexCover.

# 7.6 Trees

## VirtualTree.h

**Description:** Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S| - 1$) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.
**Time:** $\mathcal{O}(|S| \log |S|)$

"LCA.h" 9775a0, 21 lines

```
typedef vector<pair<int, int>> vpi;
vpi compressTree(LCA& lca, const vi& subset) {
  static vi rev; rev.resize(sz(lca.time));
  vi li = subset, &T = lca.time; ↑53bf4e
  auto cmp = [&](int a, int b) { return T[a] < T[b]; };
  sort(all(li), cmp);
  int m = sz(li)-1;
  rep(i,0,m) {
    int a = li[i], b = li[i+1];
    li.push_back(lca.lca(a, b));
  } ↑432667
  sort(all(li), cmp);
  li.erase(unique(all(li)), li.end());
  rep(i,0,sz(li)) rev[li[i]] = i;
  vpi ret = {pii(0, li[0])}; ↑89fc5f
  rep(i,0,sz(li)-1) {
    int a = li[i], b = li[i+1];
    ret.emplace_back(rev[lca.lca(a, b)], b);
  }
  return ret;
} ↑9775a0
```

## HLD.h

**Description:** Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most $\log n$ light edges. Supports commutative segtree modifications/queries on paths and subtrees. Customize lzseg for the desired operations. Takes as input the full adjacency list. VALS_EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0. Segment bounds passed to op are inclusive.
**Time:** $\mathcal{O}\left((\log N)^2\right)$

"../data-structures/LazySegmentTree.h" 175262, 46 lines

```
template <bool VALS_EDGES> struct HLD {
  int N, tim = 0;
  vector<vi> adj;
  vi par, siz, rt, pos;
  lzseg tree; ↑2fe83a
  HLD(vector<vi> adj_)
    : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
      rt(N),pos(N),tree(N){ dfsSz(0); dfsHld(0); }
  void dfsSz(int v) {
    for (int& u : adj[v]) {
      adj[u].erase(find(all(adj[u]), v)); ↑9a7b36
      par[u] = v;
      dfsSz(u);
      siz[v] += siz[u];
      if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
    } ↑91a138
  }
  void dfsHld(int v) {
    pos[v] = tim++; ↑d46042
    for (int u : adj[v]) {
      rt[u] = (u == adj[v][0] ? rt[v] : u);
      dfsHld(u);
    }
  } ↑c64956
  template <class B> void process(int u, int v, B op) {
    for (;; v = par[rt[v]]) {
      if (pos[u] > pos[v]) swap(u, v);
      if (rt[u] == rt[v]) break;
      op(pos[rt[v]], pos[v]);
    } ↑129d74
    op(pos[u] + VALS_EDGES, pos[v]);
  }
  void modifyPath(int u, int v, lzseg::U val) {
    process(u, v, [&](int l, int r) { tree.modify(l, r, val); }
        );
  } ↑4a426e
  lzseg::T queryPath(int u, int v) {
    lzseg::T res = tree.idT;
    process(u, v, [&](int l, int r) {
        res = tree.f(res, tree.query(l, r));
    }); ↑2c3af1
    return res;
  }
  lzseg::T querySubtree(int v) { // modifySubtree is similar
    return tree.query(pos[v] + VALS_EDGES, pos[v] + siz[v] - 1)
        ;
  }
}; ↑175262
```

## Centroid.h

**Description:** Builds the centroid tree of an undirected tree. par[$v$] is the centroid parent of $v$, or $-1$ at the root.
**Time:** $\mathcal{O}(N \log N)$, $\mathcal{O}(N)$ space.

fb6af4, 25 lines

```
struct Centroid {
  vector<vi> adj;
  vi par, siz, dead;
  Centroid(vector<vi> g)
    : adj(g), par(sz(g), -1), siz(sz(g)), dead(sz(g)) {
    if (sz(g)) build(0, -1);
  } ↑83c52f
  int dfs(int v, int p) {
    siz[v] = 1;
    for (int u : adj[v]) if (u != p && !dead[u])
      siz[v] += dfs(u, v); ↑637de9
    return siz[v];
  }
  int centroid(int v, int p, int n) {
    for (int u : adj[v]) if (u != p && !dead[u])
      if (siz[u] > n/2) return centroid(u, v, n); ↑d28e20
    return v;
  }
  void build(int v, int p) {
    int c = centroid(v, -1, dfs(v, -1)); ↑a514eb
    par[c] = p;
    dead[c] = 1;
    for (int u : adj[c]) if (!dead[u]) build(u, c);
  }
}; ↑fb6af4
```

## LinkCutTree.h

**Description:** Represents a forest of unrooted trees. You can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree.
**Time:** All operations take amortized $\mathcal{O}(\log N)$.

0fb462, 90 lines

```
struct Node { // Splay tree. Root's pp contains tree's parent.
  Node *p = 0, *pp = 0, *c[2];
  bool flip = 0;
  Node() { c[0] = c[1] = 0; fix(); } ↑210611
  void fix() {
    if (c[0]) c[0]->p = this;
    if (c[1]) c[1]->p = this;
    // (+ update sum of subtree elements etc. if wanted)
  } ↑4268f1
  void pushFlip() {
    if (!flip) return;
```

```
    flip = 0; swap(c[0], c[1]);
    if (c[0]) c[0]->flip ^= 1;
    if (c[1]) c[1]->flip ^= 1;
  } ↑d94cfc
  int up() { return p ? p->c[1] == this : -1; }
  void rot(int i, int b) {
    int h = i ^ b;
    Node *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ? y : x;
    if ((y->p = p)) p->c[up()] = y; ↑679f6a
    c[i] = z->c[i ^ 1];
    if (b < 2) {
      x->c[h] = y->c[h ^ 1];
      y->c[h ^ 1] = x;
    } ↑653614
    z->c[i ^ 1] = this;
    fix(); x->fix(); y->fix();
    if (p) p->fix();
    swap(pp, y->pp);
  } ↑966070
  void splay() {
    for (pushFlip(); p; ) {
      if (p->p) p->p->pushFlip();
      p->pushFlip(); pushFlip();
      int c1 = up(), c2 = p->up(); ↑3ef089
      if (c2 == -1) p->rot(c1, 2);
      else p->p->rot(c2, c1 != c2);
    }
  } ↑5a4303
  Node* first() {
    pushFlip();
    return c[0] ? c[0]->first() : (splay(), this);
  }
}; ↑225109

struct LinkCut {
  vector<Node> node;
  LinkCut(int N) : node(N) {}

  void link(int u, int v) { // add an edge (u, v)
    assert(!connected(u, v)); ↑661716
    makeRoot(&node[u]);
    node[u].pp = &node[v];
  } ↑557426
  void cut(int u, int v) { // remove an edge (u, v)
    Node *x = &node[u], *top = &node[v];
    makeRoot(top); x->splay();
    assert(top == (x->pp ?: x->c[0]));
    if (x->pp) x->pp = 0; ↑33e021
    else {
      x->c[0] = top->p = 0;
      x->fix();
    }
  } ↑47de4b
  bool connected(int u, int v) { // are u, v in the same tree?
    Node* nu = access(&node[u])->first();
    return nu == access(&node[v])->first();
  } ↑6fb75d
  void makeRoot(Node* u) {
    access(u);
    u->splay();
    if(u->c[0]) {
      u->c[0]->p = 0;
      u->c[0]->flip ^= 1;
      u->c[0]->pp = u; ↑3ba226
      u->c[0] = 0;
      u->fix();
    }
  } ↑9af643
  Node* access(Node* u) {
    u->splay();
    while (Node* pp = u->pp) {
      pp->splay(); u->pp = 0;
      if (pp->c[1]) {
        pp->c[1]->p = 0; pp->c[1]->pp = pp; } ↑f9babf
      pp->c[1] = u; pp->fix(); u = pp;
    }
    return u;
  }
}; ↑0fb462
```

DirectedMST.h

**Description:** Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.

**Time:** $\mathcal{O}(E \log V)$

"../data-structures/UnionFindRollback.h" 39e620, 60 lines

```
struct Edge { int a, b; ll w; };
struct Node {
  Edge key;
  Node *l, *r;
  ll delta; ↑59f245
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  } ↑31f792
  Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
  if (!a || !b) return a ?: b; ↑6b68b8
  a->prop(), b->prop();
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = merge(b, a->r)));
  return a;
} ↑5e360c
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
  RollbackUF uf(n);
  vector<Node*> heap(n); ↑a7352a
  for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
  ll res = 0;
  vi seen(n, -1), path(n), par(n);
  seen[r] = r;
  vector<Edge> Q(n), in(n, {-1,-1}), comp; ↑c7b0b9
  deque<tuple<int, int, vector<Edge>>> cycs;
  rep(s,0,n) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
      if (!heap[u]) return {-1,{}}; ↑2158f1
      Edge e = heap[u]->top();
      heap[u]->delta -= e.w, pop(heap[u]);
      Q[qi] = e, path[qi++] = u, seen[u] = s;
      res += e.w, u = uf.find(e.a); ↑fcb967
      if (seen[u] == s) {
        Node* cyc = 0;
        int end = qi, time = uf.time();
        do cyc = merge(cyc, heap[w = path[--qi]]);
        while (uf.join(u, w));
        u = uf.find(u), heap[u] = cyc, seen[u] = -1; ↑3eb5cd
        cycs.push_front({u, time, {&Q[qi], &Q[end]}});
      }
    }
    rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
  } ↑f2bc30

  for (auto& [u,t,comp] : cycs) { // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.b)] = e; ↑a32e6d
    in[uf.find(inEdge.b)] = inEdge;
  }
  rep(i,0,n) par[i] = in[i].a;
  return {res, par};
} ↑39e620
```

## 7.7 Math

### 7.7.1 Number of Spanning Trees

Create an $N \times N$ matrix `mat`, and for each edge $a \to b \in G$, do `mat[a][b]--, mat[b][b]++` (and `mat[b][a]--, mat[a][a]++` if $G$ is undirected). Remove the $i$th row and column and take the determinant; this yields the number of directed spanning trees rooted at $i$ (if $G$ is undirected, remove any row/column).

### 7.7.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \cdots \geq d_n$ exists iff $d_1 + \cdots + d_n$ is even and for every $k = 1 \ldots n$,

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

# Geometry (8)

## 8.1 Geometric primitives

Point.h

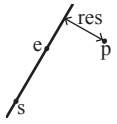
**Description:** Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

47ec0a, 28 lines

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
  typedef Point P;
  T x, y; ↑fa79fb
  explicit Point(T x=0, T y=0) : x(x), y(y) {}
  bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
  bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); } ↑189cbc
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; } ↑7ecfd2
  T cross(P a, P b) const { return (a-*this).cross(b-*this); }
  T dist2() const { return x*x + y*y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); } ↑cc70a2
  P unit() const { return *this/dist(); } // makes dist()=1
  P perp() const { return P(-y, x); } // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); } ↑e458d5
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")"; }
}; ↑47ec0a
```

## LineDistance.h

**Description:**
Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from $a$ towards $b$. $a = b$ gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.

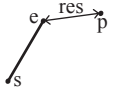



"Point.h" 53630a, 4 lines

```
template<class P>
double lineDist(P a, P b, P p) {
  return (double)(b-a).cross(p-a)/(b-a).dist();
}
```
↑53630a

## SegmentDistance.h

**Description:**
Returns the shortest distance between point p and the line segment from point s to e.



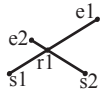

**Usage:**
```
Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;
```

"Point.h" d5f39f, 6 lines

```
typedef Point<double> P;
double segDist(P s, P e, P p) {
  if (s==e) return (p-s).dist();
  auto d = (e-s).dist2(), t = min(d,max(.0,(p-s).dot(e-s)));
  return ((p-s)*d-(e-s)*t).dist()/d;
}
```
↑d5f39f

## SegmentIntersection.h

**Description:**
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.



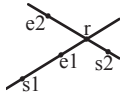

**Usage:**
```
vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;
```

"Point.h", "OnSegment.h" 9d57f2, 13 lines

```
template<class P> vector<P> segInter(P a, P b, P c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(d, b),
       oc = a.cross(b, c), od = a.cross(b, d);
  // Checks if intersection is single non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
}
```
↑ab16eb ↑64d2c1 ↑9d57f2

## LineIntersection.h

**Description:**
If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.




**Usage:**
```
auto res = lineInter(s1,e1,s2,e2);
if (res.f == 1)
cout << "intersection point at " << res.s << endl;
```

"Point.h" a01f81, 8 lines

```
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0) // if parallel
    return {-(s1.cross(e1, s2) == 0), P(0, 0)};
  auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
  return {1, (s1 * p + e1 * q) / d};
}
```
↑47e53e ↑a01f81

## SideOf.h

**Description:** Returns where $p$ is as seen from $s$ towards $e$. 1/0/-1 $\Leftrightarrow$ left/on line/right. If the optional argument $eps$ is given 0 is returned if $p$ is within distance $eps$ from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

**Usage:**
```
bool left = sideOf(p1,p2,q)==1;
```

"Point.h" ca4665, 9 lines

```
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }

template<class P>
int sideOf(P s, P e, P p, double eps) {
  auto a = (e-s).cross(p-s);
  double l = (e-s).dist()*eps;
  return (a > l) - (a < -l);
}
```
↑914322 ↑ca4665

## OnSegment.h

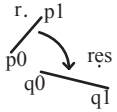
**Description:** Returns true iff p lies on the line segment from s to e. Use (segDist(s,e,p)<=epsilon) instead when using Point<double>.

"Point.h" c597e8, 3 lines

```
template<class P> bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```
↑c597e8

## LinearTransformation.h

**Description:**
Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.




"Point.h" 7871e9, 5 lines

```
typedef Point<double> P;
P linearTransformation(P p0, P p1, P q0, P q1, P r) {
  P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
  return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
}
```
↑7871e9

## LineProjectionReflection.h

**Description:** Projects point p onto line ab. Set refl=true to get reflection of point p across line ab instead. The wrong point will be returned if P is an integer point and the desired point doesn't have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow.

"Point.h" b5562d, 5 lines

```
template<class P>
P lineProj(P a, P b, P p, bool refl=false) {
  P v = b - a;
  return p - v.perp()*(1+refl)*v.cross(p-a)/v.dist2();
}
```
↑b5562d

## Angle.h

**Description:** A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

**Usage:**
```
vector<Angle> v = {w[0], w[0].t360() ...}; // sorted
int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }
// sweeps j such that (j-i) represents the number of positively
oriented triangles with vertices at 0 and i
```

0f0602, 35 lines

```
struct Angle {
  int x, y;
  int t;
  Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}
  Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }
  int half() const {
    assert(x || y);
    return y < 0 || (y == 0 && x < 0);
  }
  Angle t90() const { return {-y, x, t + (half() && x >= 0)}; }
  Angle t180() const { return {-x, -y, t + half()}; }
  Angle t360() const { return {x, y, t + 1}; }
};
bool operator<(Angle a, Angle b) {
  // add a.dist2() and b.dist2() to also compare distances
  return make_tuple(a.t, a.half(), a.y * (ll)b.x) <
         make_tuple(b.t, b.half(), a.x * (ll)b.y);
}

// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
  if (b < a) swap(a, b);
  return (b < a.t180() ?
          make_pair(a, b) : make_pair(b, a.t360()));
}
Angle operator+(Angle a, Angle b) { // point a + vector b
  Angle r(a.x + b.x, a.y + b.y, a.t);
  if (a.t180() < r) r.t--;
  return r.t180() < a ? r.t360() : r;
}
Angle angleDiff(Angle a, Angle b) { // angle b - angle a
  int tu = b.t - a.t; a.t = b.t;
  return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)};
}
```
↑e258c0 ↑e78926 ↑eccd19 ↑3fb429 ↑0f0602

# 8.2 Circles

## CircleIntersection.h

**Description:** Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"Point.h" 84d6d3, 11 lines

```
typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out) {
  if (a == b) { assert(r1 != r2); return false; }
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
         p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
  if (sum*sum < d2 || dif*dif > d2) return false;
  P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
  *out = {mid + per, mid - per};
  return true;
}
```
↑2e6973 ↑84d6d3

## CircleTangents.h

**Description:** Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case `.f = .s` and the tangent line is perpendicular to the line between the centers). .f and .s give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

"Point.h" b0153d, 13 lines

```
template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
  P d = c2 - c1;
  double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
  if (d2 == 0 || h2 < 0)  return {};  ↑c18727
  vector<pair<P, P>> out;
  for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
  }  ↑41b560
  if (h2 == 0) out.pop_back();
  return out;
}  ↑b0153d
```

## CircleLine.h

**Description:** Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. P is intended to be Point<double>.

"Point.h" e0cfba, 9 lines

```
template<class P>
vector<P> circleLine(P c, double r, P a, P b) {
  P ab = b - a, p = a + ab * (c-a).dot(ab) / ab.dist2();
  double s = a.cross(b, c), h2 = r*r - s*s / ab.dist2();
  if (h2 < 0) return {};  ↑64a27f
  if (h2 == 0) return {p};
  P h = ab.unit() * sqrt(h2);
  return {p - h, p + h};
}  ↑e0cfba
```

## CirclePolygonIntersection.h

**Description:** Returns the area of the intersection of a circle with a ccw polygon.
**Time:** $\mathcal{O}(n)$

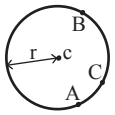
"Point.h" 19add1, 19 lines

```
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
  auto tri = [&](P p, P q) {
    auto r2 = r * r / 2;  ↑a6cf13
    P d = q - p;
    auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2();
    auto det = a * a - b;
    if (det <= 0) return arg(p, q) * r2;  ↑3710c6
    auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
    if (t < 0 || 1 <= s) return arg(p, q) * r2;
    P u = p + d * s, v = q + d * (t-1);
    return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;
  };  ↑e30856
  auto sum = 0.0;
  rep(i,0,sz(ps))
    sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
  return sum;
}  ↑19add1
```

## Circumcircle.h

**Description:**

The circumcirle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.




"Point.h" 6cffc5, 9 lines

```
typedef Point<double> P;
double ccRadius(P A, P B, P C) {
  return (B-A).dist()*(C-B).dist()*(A-C).dist()/
      abs((B-A).cross(C-A))/2;
}  ↑d0b6d9
P ccCenter(P A, P B, P C) {
  P b = C-A, c = B-A;
  return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}  ↑6cffc5
```

## MinimumEnclosingCircle.h

**Description:** Computes the minimum circle that encloses a set of points.
**Time:** expected $\mathcal{O}(n)$

"Circumcircle.h" 09dd0a, 17 lines

```
pair<P, double> mec(vector<P> ps) {
  shuffle(all(ps), mt19937(time(0)));
  P o = ps[0];
  double r = 0, EPS = 1 + 1e-8;  ↑56a5f0
  rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
    o = ps[i], r = 0;
    rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
      o = (ps[i] + ps[j]) / 2;
      r = (o - ps[i]).dist();  ↑da034d
      rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
        o = ccCenter(ps[i], ps[j], ps[k]);
        r = (o - ps[i]).dist();
      }
    }
  }
  return {o, r};
}  ↑09dd0a
```

# 8.3 Polygons

## InsidePolygon.h

**Description:** Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.
**Usage:** `vector<P> v = {P{4,4}, P{1,2}, P{2,1}};`
`bool in = inPolygon(v, P{3, 3}, false);`
**Time:** $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h" 2bf504, 11 lines

```
template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
  int cnt = 0, n = sz(p);
  rep(i,0,n) {
    P q = p[(i + 1) % n];  ↑1c161f
    if (onSegment(p[i], q, a)) return !strict;
    //or: if (segDist(p[i], q, a) <= eps) return !strict;
    cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
  }
  return cnt;
}  ↑2bf504
```

## PolygonArea.h

**Description:** Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h" f12300, 6 lines

```
template<class T>
T polygonArea2(vector<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
  return a;
}  ↑f12300
```

## PolygonCenter.h

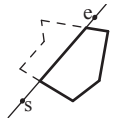
**Description:** Returns the center of mass for a polygon.
**Time:** $\mathcal{O}(n)$

"Point.h" 9706dc, 9 lines

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
  P res(0, 0); double A = 0;
  for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
    res = res + (v[i] + v[j]) * v[j].cross(v[i]);  ↑082251
    A += v[j].cross(v[i]);
  }
  return res / A / 3;
}  ↑9706dc
```

## PolygonCut.h

**Description:**
Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.
**Usage:** `vector<P> p = ...;`
`p = polygonCut(p, P(0,0), P(1,0));`

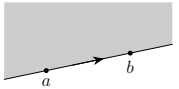



"Point.h" d07181, 13 lines

```
typedef Point<double> P;
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
  vector<P> res;
  rep(i,0,sz(poly)) {
    P cur = poly[i], prev = i ? poly[i-1] : poly.back();
    auto a = s.cross(e, cur), b = s.cross(e, prev);  ↑fb8c41
    if ((a < 0) != (b < 0))
      res.push_back(cur + (prev - cur) * (a / (a - b)));
    if (a < 0)
      res.push_back(cur);  ↑b8d2b2
  }
  return res;
}  ↑d07181
```

## HalfPlaneIntersection.h

**Description:**
Intersects closed half-planes to the left of directed lines $a \to b$, with $a \neq b$. The intersection must be bounded (add bounding box edges if needed). Returns its vertices in counter-clockwise order, or empty if empty or of area at most eps. Nearly parallel lines can cause precision issues.
**Usage:** `hpInter({HP(P(0,0), P(1,0)), ...});`
**Time:** $\mathcal{O}(n \log n)$




"Point.h" d529db, 40 lines

```
typedef Point<double> P;
const double eps = 1e-9;
struct HP {
  P p, d;
  HP(P a, P b) : p(a), d((b-a).unit()) {}  ↑e70422
  int half() const { return d.y < 0 || (d.y == 0 && d.x < 0); }
  bool operator<(HP b) const {
    return half() != b.half() ? half() < b.half() : d.cross(b.d
        ) > 0;
  }  ↑7a5080
  bool out(P q) { return d.cross(q-p) < -eps; }
  bool parallel(HP b) { return abs(d.cross(b.d)) < eps; }
  P inter(HP b) { return p + d * (b.d.cross(p-b.p) / d.cross(b.
      d)); }
};  ↑242311

vector<P> hpInter(vector<HP> h) {
  sort(all(h));
```

```
  while (sz(h) > 1 && h[0].parallel(h.back()) && h[0].d.dot(h.
      back().d) > 0) {
    if (h[0].d.cross(h.back().p-h[0].p) > 0) h[0] = h.back();
    h.pop_back();
  } ↑1507b4
  deque<HP> q;
  for (HP a : h) {
    while (sz(q) > 1 && a.out(q[sz(q)-2].inter(q.back()))) q.
        pop_back();
    while (sz(q) > 1 && a.out(q[0].inter(q[1]))) q.pop_front();
    if (!q.empty() && a.parallel(q.back())) {
      if (a.d.dot(q.back().d) < 0) return {}; ↑0f7c75
      if (a.d.cross(q.back().p-a.p) >= 0) continue;
      q.pop_back();
    } ↑a7a854
    q.push_back(a);
  }
  while (sz(q) > 2 && q[0].out(q[sz(q)-2].inter(q.back()))) q.
      pop_back();
  while (sz(q) > 2 && q.back().out(q[0].inter(q[1]))) q.
      pop_front();
  if (sz(q) < 3 || q[0].parallel(q.back())) return {};
  vector<P> res; ↑dd6649
  rep(i,0,sz(q)) res.push_back(q[i].inter(q[(i+1)%sz(q)]));
  double area = 0;
  rep(i,1,sz(res)-1) area += res[0].cross(res[i], res[i+1]);
  return area > 2 * eps ? res : vector<P>{};
} ↑d529db
```

## PolygonUnion.h

**Description:** Calculates the area of the union of $n$ polygons (not necessarily convex). The points within each polygon must be given in CCW order. (Epsilon checks may optionally be added to sideOf/sgn, but shouldn't be needed.)

**Time:** $\mathcal{O}\left(N^2\right)$, where $N$ is the total number of points

"Point.h", "SideOf.h" 5ffb66, 33 lines

```
typedef Point<double> P;
double rat(P a, P b) { return sgn(b.x) ? a.x/b.x : a.y/b.y; }
double polyUnion(vector<vector<P>>& poly) {
  double ret = 0; ↑9680ea
  rep(i,0,sz(poly)) rep(v,0,sz(poly[i])) {
    P A = poly[i][v], B = poly[i][(v + 1) % sz(poly[i])];
    vector<pair<double, int>> segs = {{0, 0}, {1, 0}};
    rep(j,0,sz(poly)) if (i != j) {
      rep(u,0,sz(poly[j])) {
        P C = poly[j][u], D = poly[j][(u + 1) % sz(poly[j])];
        int sc = sideOf(A, B, C), sd = sideOf(A, B, D);
        if (sc != sd) {
          double sa = C.cross(D, A), sb = C.cross(D, B);
          if (min(sc, sd) < 0)
            segs.emplace_back(sa / (sa - sb), sgn(sc - sd));
        } else if (!sc && !sd && j<i && sgn((B-A).dot(D-C))>0){
          segs.emplace_back(rat(C - A, B - A), 1); ↑a4636e
          segs.emplace_back(rat(D - A, B - A), -1);
        }
      } ↑c4b419
    }
    sort(all(segs));
    for (auto& s : segs) s.f = min(max(s.f, 0.0), 1.0);
    double sum = 0;
    int cnt = segs[0].s; ↑dda26a
    rep(j,1,sz(segs)) {
      if (!cnt) sum += segs[j].f - segs[j - 1].f;
      cnt += segs[j].s;
    } ↑0b2fdc
    ret += A.cross(B) * sum;
  }
  return ret / 2;
} ↑5ffb66
```

## ConvexHull.h



**Description:**
Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.

**Time:** $\mathcal{O}\left(n \log n\right)$

"Point.h" 310954, 13 lines

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts));
  vector<P> h(sz(pts)+1); ↑3e3497
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
      h[t++] = p; ↑f4a7b9
    }
  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
} ↑310954
```

## MinkowskiSum.h

**Description:** Minkowski sum of two convex hulls in counter-clockwise order, without repeated or redundant collinear vertices (as returned by convexHull). Returns counter-clockwise vertices starting at the lowest $(y, x)$. Empty hulls, points and segments are supported. Arithmetic must fit in ll.

**Time:** $\mathcal{O}\left(n + m\right)$

"Point.h" 61d8b9, 19 lines

```
typedef Point<ll> P;
vector<P> minkowski(vector<P> a, vector<P> b) {
  int n = sz(a), m = sz(b), i = 0, j = 0;
  if (!n || !m) return {}; ↑f12f2c
  for (auto p : {&a, &b}) {
    auto cmp = [](P u, P v) { return tie(u.y,u.x) < tie(v.y,v.x
        ); };
    rotate(p->begin(), min_element(p->begin(), p->end(), cmp),
        p->end());
    p->push_back((*p)[0]);
  } ↑2ba4da
  vector<P> res;
  while (i < n || j < m) {
    res.push_back(a[i] + b[j]);
    int c = i == n ? -1 : j == m ? 1 :
      sgn((a[i+1] - a[i]).cross(b[j+1] - b[j]));
    i += c >= 0; ↑4b0e90
    j += c <= 0;
  }
  return res;
} ↑61d8b9
```

## HullDiameter.h

**Description:** Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).

**Time:** $\mathcal{O}\left(n\right)$

"Point.h" 7acf46, 12 lines

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
  int n = sz(S), j = n < 2 ? 0 : 1;
  pair<ll, array<P, 2>> res({0, {S[0], S[0]}}); ↑12ea1a
  rep(i,0,j)
    for (;; j = (j + 1) % n) {
      res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
      if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
        break;
    }
  return res.s;
} ↑7acf46
```

## PointInsideHull.h

**Description:** Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.

**Time:** $\mathcal{O}\left(\log N\right)$

"Point.h", "SideOf.h", "OnSegment.h" 71446b, 14 lines

```
typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict = true) {
  int a = 1, b = sz(l) - 1, r = !strict; ↑3f3f6c
  if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
  if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
  if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p)<= -r)
    return false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2; ↑e79ab6
    (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
  }
  return sgn(l[a].cross(l[b], p)) < r;
} ↑71446b
```

## LineHullIntersection.h

**Description:** Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $\bullet$ $(-1,-1)$ if no collision, $\bullet$ $(i,-1)$ if touching the corner $i$, $\bullet$ $(i,i)$ if along side $(i,i+1)$, $\bullet$ $(i,j)$ if crossing sides $(i,i+1)$ and $(j,j+1)$. In the last case, if a corner $i$ is crossed, this is treated as happening on side $(i,i+1)$. The points are returned in the same order as the line hits the polygon. `extrVertex` returns the point of a hull with the max projection onto a line.

**Time:** $\mathcal{O}\left(\log n\right)$

"Point.h" 7cf45b, 39 lines

```
#define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P> int extrVertex(vector<P>& poly, P dir) {
  int n = sz(poly), lo = 0, hi = n;
  if (extr(0)) return 0; ↑b9df6a
  while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
    (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
  } ↑efd609
  return lo;
}

#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
  int endA = extrVertex(poly, (a - b).perp());
  int endB = extrVertex(poly, (b - a).perp()); ↑bc546b
  if (cmpL(endA) < 0 || cmpL(endB) > 0)
    return {-1, -1};
  array<int, 2> res;
  rep(i,0,2) {
    int lo = endB, hi = endA, n = sz(poly); ↑090d37
    while ((lo + 1) % n != hi) {
      int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
      (cmpL(m) == cmpL(endB) ? lo : hi) = m;
    }
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
  } ↑d56a85
  if (res[0] == res[1]) return {res[0], -1};
  if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
      case 0: return {res[0], res[0]}; ↑ab4398
      case 2: return {res[1], res[1]};
```

```
    }
  return res;
} ↑7cf45b
```

## 8.4 Misc. Point Set Problems

### ClosestPair.h

**Description:** Finds the closest pair of points.
**Time:** $\mathcal{O}(n \log n)$

"Point.h" bfee40, 17 lines

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y < b.y; }); ↑9e7fdf
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.f), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]); ↑724c9e
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
  return ret.s;
} ↑bfee40
```

### ManhattanMST.h

**Description:** Given $N$ points, returns up to $4N$ edges, which are guaranteed to contain a minimum spanning tree for the graph with edge weights $w(p, q) = |p.x - q.x| + |p.y - q.y|$. Edges are in the form (distance, src, dst). Use a standard MST algorithm on the result to find the final MST.
**Time:** $\mathcal{O}(N \log N)$

"Point.h" 6b4a71, 23 lines

```
typedef Point<int> P;
vector<array<int, 3>> manhattanMST(vector<P> ps) {
  vi id(sz(ps));
  iota(all(id), 0);
  vector<array<int, 3>> edges; ↑bded47
  rep(k,0,4) {
    sort(all(id), [&](int i, int j) {
         return (ps[i]-ps[j]).x < (ps[j]-ps[i]).y;});
    map<int, int> sweep; ↑0a2d30
    for (int i : id) {
      for (auto it = sweep.lower_bound(-ps[i].y);
                it != sweep.end(); sweep.erase(it++)) {
        int j = it->s;
        P d = ps[i] - ps[j];
        if (d.y > d.x) break; ↑14756b
        edges.push_back({d.y + d.x, i, j});
      }
      sweep[-ps[i].y] = i;
    } ↑b5ba59
    for (P& p : ps) if (k & 1) p.x = -p.x; else swap(p.x, p.y);
  }
  return edges;
} ↑6b4a71
```

### KDTree.h

**Description:** KD-tree (2d, can be extended to 3d)

"Point.h" b934ef, 63 lines

```
typedef long long T;
typedef Point<T> P;
const T INF = numeric_limits<T>::max();

bool on_x(P a, P b) { return a.x < b.x; } ↑f3f362
bool on_y(P a, P b) { return a.y < b.y; }

struct Node {
  P pt; // if this is a leaf, the single point in it
  T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds
  Node *first = 0, *second = 0; ↑97536a

  T distance(P p) { // min squared distance to a point
    T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
    T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y); ↑27f2f9
    return (P(x,y) - p).dist2();
  }

  Node(vector<P>&& vp) : pt(vp[0]) {
    for (P p : vp) {
      x0 = min(x0, p.x); x1 = max(x1, p.x); ↑d41a77
      y0 = min(y0, p.y); y1 = max(y1, p.y);
    }
    if (vp.size() > 1) {
      // split on x if width >= height (not ideal...)
      sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);
      // divide by taking half the array for each child (not
      // best performance with many duplicates in the middle)
      int half = sz(vp)/2;
      first = new Node({vp.begin(), vp.begin() + half});
      second = new Node({vp.begin() + half, vp.end()});
    } ↑a6ca3e
  }
};

struct KDTree {
  Node* root; ↑14d774
  KDTree(const vector<P>& vp) : root(new Node({all(vp)})) {}

  pair<T, P> search(Node *node, P p) {
    if (!node->first) {
      // uncomment if we should not find the point itself:
      // if (p == node->pt) return {INF, P()};
      return make_pair((p - node->pt).dist2(), node->pt);
    } ↑5e2897

    Node *f = node->first, *s = node->second;
    T bfirst = f->distance(p), bsec = s->distance(p);
    if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);

    // search closest side first, other side if needed
    auto best = search(f, p); ↑76ca18
    if (bsec < best.f)
      best = min(best, search(s, p));
    return best;
  } ↑0a43c4

  // find nearest point to a point, and its squared distance
  // (requires an arbitrary operator< for Point)
  pair<T, P> nearest(P p) {
    return search(root, p);
  }
}; ↑b934ef
```

### FastDelaunay.h

**Description:** Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order $\{t[0][0], t[0][1], t[0][2], t[1][0], \dots\}$, all counter-clockwise.
**Time:** $\mathcal{O}(n \log n)$

"Point.h" 66aee7, 88 lines

```
typedef Point<ll> P;
typedef struct Quad* Q;
typedef __int128_t lll; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX,LLONG_MAX); // not equal to any other point

struct Quad {
  Q rot, o; P p = arb; bool mark; ↑3fba49
  P& F() { return r()->p; }
  Q& r() { return rot->rot; }
  Q prev() { return rot->o->rot; } ↑a6d183
  Q next() { return r()->prev(); }
} *H;

bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
  lll p2 = p.dist2(), A = a.dist2()-p2,
      B = b.dist2()-p2, C = c.dist2()-p2; ↑4e353f
  return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
}
Q makeEdge(P orig, P dest) {
  Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
  H = r->o; r->r()->r() = r; ↑603488
  rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
  r->p = orig; r->F() = dest;
  return r;
} ↑25ccf6
void splice(Q a, Q b) {
  swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
} ↑0ef350
Q connect(Q a, Q b) {
  Q q = makeEdge(a->F(), b->p);
  splice(q, a->next());
  splice(q->r(), b);
  return q;
} ↑eef885

pair<Q,Q> rec(const vector<P>& s) {
  if (sz(s) <= 3) {
    Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
    if (sz(s) == 2) return { a, a->r() };
    splice(a->r(), b); ↑f1433b
    auto side = s[0].cross(s[1], s[2]);
    Q c = side ? connect(b, a) : 0;
    return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
  } ↑60c127

#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
  Q A, B, ra, rb;
  int half = sz(s) / 2;
  tie(ra, A) = rec({all(s) - half}); ↑576005
  tie(B, rb) = rec({sz(s) - half + all(s)});
  while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
         (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
  Q base = connect(B->r(), A);
  if (A->p == ra->p) ra = base->r(); ↑ad8016
  if (B->p == rb->p) rb = base;

#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
      Q t = e->dir; \
      splice(e, e->prev()); \
      splice(e->r(), e->r()->prev()); \
      e->o = H; H = e; e = t; \
    }
  for (;;) {
    DEL(LC, base->r(), o);  DEL(RC, base, prev()); ↑9b353c
    if (!valid(LC) && !valid(RC)) break;
    if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
      base = connect(RC, base->r());
    else
      base = connect(base->r(), LC->r());
  } ↑17ceb8
```

```
  return { ra, rb };
}

vector<P> triangulate(vector<P> pts) {
  sort(all(pts));  assert(unique(all(pts)) == pts.end());
  if (sz(pts) < 2) return {};
  Q e = rec(pts).f;
  vector<Q> q = {e}; ↑1cb239
  int qi = 0;
  while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
  q.push_back(c->r()); c = c->next(); } while (c != e); }
  ADD; pts.clear(); ↑e8ee6e
  while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;
  return pts;
} ↑66aee7
```

## 8.5 3D

### PolyhedronVolume.h

**Description:** Magic formula for the volume of a polyhedron. Faces should point outwards.

3058c3, 6 lines

```
template<class V, class L>
double signedPolyVolume(const V& p, const L& trilist) {
  double v = 0;
  for (auto i : trilist) v += p[i.a].cross(p[i.b]).dot(p[i.c]);
  return v / 6;
} ↑3058c3
```

### Point3D.h

**Description:** Class to handle points in 3D space. T can be e.g. double or long long.

a94b7c, 31 lines

```
template<class T> struct Point3D {
  typedef Point3D P;
  T x, y, z;
  explicit Point3D(T x=0, T y=0, T z=0) : x(x), y(y), z(z) {}
  bool operator<(P p) const {
    return tie(x, y, z) < tie(p.x, p.y, p.z); } ↑6f4a4d
  bool operator==(P p) const {
    return tie(x, y, z) == tie(p.x, p.y, p.z); }
  P operator+(P p) const { return P(x+p.x, y+p.y, z+p.z); }
  P operator-(P p) const { return P(x-p.x, y-p.y, z-p.z); }
  P operator*(T d) const { return P(x*d, y*d, z*d); }
  P operator/(T d) const { return P(x/d, y/d, z/d); }
  T dot(P p) const { return x*p.x + y*p.y + z*p.z; }
  P cross(P p) const {
    return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x);
  } ↑ba7fe7
  T dist2() const { return x*x + y*y + z*z; }
  double dist() const { return sqrt((double)dist2()); }
  //Azimuthal angle (longitude) to x-axis in interval [-pi, pi]
  double phi() const { return atan2(y, x); } ↑e19b65
  //Zenith angle (latitude) to the z-axis in interval [0, pi]
  double theta() const { return atan2(sqrt(x*x+y*y),z); }
  P unit() const { return *this/(T)dist(); } //makes dist()=1
  //returns unit vector normal to *this and p
  P normal(P p) const { return cross(p).unit(); } ↑098ef5
  //returns point rotated 'angle' radians ccw around axis
  P rotate(double angle, P axis) const {
    double s = sin(angle), c = cos(angle); P u = axis.unit();
    return u*dot(u)*(1-c) + (*this)*c - cross(u)*s;
  }
}; ↑a94b7c
```

### Hull3D.h

**Description:** Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.

**Time:** $\mathcal{O}(n^2)$

"Point3D.h" 5b45fc, 49 lines

```
typedef Point3D<double> P3;

struct PR {
  void ins(int x) { (a == -1 ? a : b) = x; }
  void rem(int x) { (a == x ? a : b) = -1; } ↑e28e42
  int cnt() { return (a != -1) + (b != -1); }
  int a, b;
}; ↑9a9457

struct F { P3 q; int a, b, c; };

vector<F> hull3d(const vector<P3>& A) {
  assert(sz(A) >= 4);
  vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1}));
#define E(x,y) E[f.x][f.y]
  vector<F> FS; ↑6eec88
  auto mf = [&](int i, int j, int k, int l) {
    P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
    if (q.dot(A[l]) > q.dot(A[i]))
      q = q * -1; ↑d6434b
    F f{q, i, j, k};
    E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
    FS.push_back(f);
  }; ↑f13ccf
  rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4)
    mf(i, j, k, 6 - i - j - k);

  rep(i,4,sz(A)) {
    rep(j,0,sz(FS)) {
      F f = FS[j]; ↑77d954
      if(f.q.dot(A[i]) > f.q.dot(A[f.a])) {
        E(a,b).rem(f.c);
        E(a,c).rem(f.b);
        E(b,c).rem(f.a);
        swap(FS[j--], FS.back()); ↑2eb5b4
        FS.pop_back();
      }
    } ↑66122d
    int nw = sz(FS);
    rep(j,0,nw) {
      F f = FS[j];
#define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.c);
      C(a, b, c); C(a, c, b); C(b, c, a);
    }
  } ↑c8c803
  for (F& it : FS) if ((A[it.b] - A[it.a]).cross(
    A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
  return FS;
}; ↑5b45fc
```

### SphericalDistance.h

**Description:** Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 ($\phi_1$) and f2 ($\phi_2$) from x axis and zenith angles (latitude) t1 ($\theta_1$) and t2 ($\theta_2$) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx · radius is then the difference between the two points in the $x$ direction and $d$ · radius is the total distance between the points.

611f07, 8 lines

```
double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius) {
  double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
  double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
  double dz = cos(t2) - cos(t1); ↑6da400
  double d = sqrt(dx*dx + dy*dy + dz*dz);
  return radius*2*asin(d/2);
} ↑611f07
```

# Strings (9)

### KMP.h

**Description:** pi[$x$] computes the length of the longest prefix of $s$ that ends at $x$, other than $s[0 \ldots x]$ itself (abacaba → 0010123). Can be used to find all occurrences of a string.

**Time:** $\mathcal{O}(n)$

d4375c, 16 lines

```
vi pi(const string& s) {
  vi p(sz(s));
  rep(i,1,sz(s)) {
    int g = p[i-1];
    while (g && s[i] != s[g]) g = p[g-1]; ↑f6d6b9
    p[i] = g + (s[i] == s[g]);
  }
  return p;
} ↑9cb7fc

vi match(const string& s, const string& pat) {
  vi p = pi(pat + '\0' + s), res;
  rep(i,sz(p)-sz(s),sz(p))
    if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
  return res;
} ↑d4375c
```

### ZFunc.h

**Description:** $z[i]$ computes the length of the longest common prefix of $s[i:]$ and $s$, except $z[0] = 0$. (abacaba → 0010301)

**Time:** $\mathcal{O}(n)$

ee09e2, 12 lines

```
vi Z(const string& S) {
  vi z(sz(S));
  int l = -1, r = -1;
  rep(i,1,sz(S)) {
    z[i] = i >= r ? 0 : min(r - i, z[i - l]); ↑fc3afa
    while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
      z[i]++;
    if (i + z[i] > r)
      l = i, r = i + z[i];
  }
  return z;
} ↑ee09e2
```

### Manacher.h

**Description:** For each position in a string, computes $p[0][i]$ = half length of longest even palindrome around pos $i$, $p[1][i]$ = longest odd (half rounded down).

**Time:** $\mathcal{O}(N)$

e7ad79, 13 lines

```
array<vi, 2> manacher(const string& s) {
  int n = sz(s);
  array<vi,2> p = {vi(n+1), vi(n)};
  rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
    int t = r-i+!z; ↑510161
    if (i<r) p[z][i] = min(t, p[z][l+t]);
    int L = i-p[z][i], R = i+p[z][i]-!z;
    while (L>=1 && R+1<n && s[L-1] == s[R+1])
      p[z][i]++, L--, R++;
    if (R>r) l=L, r=R; ↑168507
  }
  return p;
} ↑e7ad79
```

## MinRotation.h

**Description:** Finds the lexicographically smallest rotation of a string.
**Usage:** `rotate(v.begin(), v.begin()+minRotation(v), v.end());`
**Time:** $\mathcal{O}(N)$

d07a42, 8 lines

```
int minRotation(string s) {
  int a=0, N=sz(s); s += s;
  rep(b,0,N) rep(k,0,N) {
    if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1); break;}
    if (s[a+k] > s[b+k]) { a = b; break; } ↑20f912
  }
  return a;
} ↑d07a42
```

## SuffixArray.h

**Description:** Builds suffix array for a string. `sa[i]` is the starting index of the suffix which is $i$'th in the sorted suffix array. The returned vector is of size $n+1$, and `sa[0] = n`. The `lcp` array contains longest common prefixes for neighbouring strings in the suffix array: `lcp[i] = lcp(sa[i], sa[i-1])`, `lcp[0] = 0`. The input string must not contain any nul chars.
**Time:** $\mathcal{O}(n \log n)$

635552, 22 lines

```
struct SuffixArray {
  vi sa, lcp;
  SuffixArray(string s, int lim=256) { // or vector<int>
    s.push_back(0); int n = sz(s), k = 0, a, b;
    vi x(all(s)), y(n), ws(max(n, lim)); ↑73df2e
    sa = lcp = y, iota(all(sa), 0);
    for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
      p = j, iota(all(y), n - j);
      rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
      fill(all(ws), 0); ↑4e8fea
      rep(i,0,n) ws[x[i]]++;
      rep(i,1,lim) ws[i] += ws[i - 1];
      for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
      swap(x, y), p = 1, x[sa[0]] = 0;
      rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
        (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
    } ↑7ff7c1
    for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
      for (k && k--, j = sa[x[i] - 1];
          s[i + k] == s[j + k]; k++);
  }
}; ↑635552
```

## SuffixTree.h

**Description:** Ukkonen's algorithm for online suffix tree construction. Each node contains indices $[l, r]$ into the string, and a list of child nodes. Suffixes are given by traversals of this tree, joining $[l, r]$ substrings. The root is 0 (has $l = -1$, $r = -1$), non-existent children are $-1$. To get a complete tree, append a dummy symbol – otherwise it may contain an incomplete path (still useful for substring matching, though).
**Time:** $\mathcal{O}(26N)$

4763e6, 50 lines

```
struct SuffixTree {
  enum { N = 200010, ALPHA = 26 }; // N ~ 2*maxlen+10
  int toi(char c) { return c - 'a'; }
  string a; // v = cur node, q = cur position
  int t[N][ALPHA],l[N],r[N],p[N],s[N],v=0,q=0,m=2; ↑b1f1b1

  void ukkadd(int i, int c) { suff:
    if (r[v]<q) {
      if (t[v][c]==-1) { t[v][c]=m;  l[m]=i;
        p[m++]=v; v=s[v]; q=r[v]+1;  goto suff; } ↑87a1bb
      v=t[v][c]; q=l[v];
    }
    if (q==-1 || c==toi(a[q])) q++; else {
      l[m+1]=i;  p[m+1]=m;  l[m]=l[v];  r[m]=q-1;
      p[m]=p[v];  t[m][c]=m+1;  t[m][toi(a[q])]=v; ↑f25056
      l[v]=q;  p[v]=m;  t[p[m]][toi(a[l[m]])]=m;
      v=s[p[m]];  q=l[m];
      while (q<=r[m]) { v=t[v][toi(a[q])];  q+=r[v]-l[v]+1; }
      if (q==r[m]+1)  s[m]=v;  else s[m]=m+2;
      q=r[v]+1-(q-r[m]-1);  m+=2;  goto suff;
    } ↑200f88
  }

  SuffixTree(string a) : a(a) {
    fill(r,r+N,sz(a)-1); ↑24676c
    memset(s, 0, sizeof s);
    memset(t, -1, sizeof t);
    fill(t[1],t[1]+ALPHA,0);
    s[0] = 1; l[0] = l[1] = -1; r[0] = r[1] = -1; p[0] = p[1] =
        0;
    rep(i,0,sz(a)) ukkadd(i, toi(a[i]));
  } ↑c9d23b

  // example: find longest common substring (uses ALPHA = 28)
  pii best; ↑a26c2a
  int lcs(int node, int i1, int i2, int olen) {
    if (l[node] <= i1 && i1 <= r[node]) return 1;
    if (l[node] <= i2 && i2 <= r[node]) return 2;
    int mask = 0, len = node ? olen + (r[node] - l[node] + 1) :
        0;
    rep(c,0,ALPHA) if (t[node][c] != -1)
      mask |= lcs(t[node][c], i1, i2, len); ↑9b65f8
    if (mask == 3)
      best = max(best, {len, r[node] - len + 1});
    return mask;
  } ↑0361e9
  static pii LCS(string s, string t) {
    SuffixTree st(s + (char)('z' + 1) + t + (char)('z' + 2));
    st.lcs(0, sz(s), sz(s) + 1 + sz(t), 0);
    return st.best;
  }
}; ↑4763e6
```

## Hashing.h

**Description:** Self-explanatory methods for string hashing.

eebd3d, 44 lines

```
// Arithmetic mod 2^64-1. 2x slower than mod 2^64 and more
// code, but works on evil test data (e.g. Thue-Morse, where
// ABBA... and BAAB... of length 2^10 hash the same mod 2^64).
// "typedef ull H;" instead if you think test data is random,
// or work mod 10^9+7 if the Birthday paradox is not a problem.
typedef uint64_t ull; ↑41d24d
struct H {
  ull x; H(ull x=0) : x(x) {}
  H operator+(H o) { return x + o.x + (x + o.x < x); }
  H operator-(H o) { return *this + ~o.x; } ↑98ccfa
  H operator*(H o) { auto m = (__uint128_t)x * o.x;
    return H((ull)m) + (ull)(m >> 64); }
  ull get() const { return x + !~x; }
  bool operator==(H o) const { return get() == o.get(); }
  bool operator<(H o) const { return get() < o.get(); }
}; ↑40d284
static const H C = (ll)1e11+3; // (order ~ 3e9; random also ok)

struct HashInterval {
  vector<H> ha, pw; ↑5588fd
  HashInterval(string& str) : ha(sz(str)+1), pw(ha) {
    pw[0] = 1;
    rep(i,0,sz(str))
      ha[i+1] = ha[i] * C + str[i],
      pw[i+1] = pw[i] * C;
  } ↑47478c
  H hashInterval(int a, int b) { // hash [a, b]
    return ha[b+1] - ha[a] * pw[b - a + 1];
  }
}; ↑29cee8

vector<H> getHashes(string& str, int length) {
  if (sz(str) < length) return {};
  H h = 0, pw = 1; ↑bdbaae
  rep(i,0,length)
    h = h * C + str[i], pw = pw * C;
  vector<H> ret = {h};
  rep(i,length,sz(str)) {
    ret.push_back(h = h * C + str[i] - pw * str[i-length]);
  } ↑e19187
  return ret;
}

H hashString(string& s){H h{}; for(char c:s) h=h*C+c;return h;}
```

## AhoCorasick.h

**Description:** Suffix link automaton trie for Aho-Corasick. Use `go` to traverse the automaton. `for (int v = u; v; v = trie.next(v))` can be used to iterate over all pattern matches at the current index. `go`, `next`, and `get_link` are amortized $\mathcal{O}(1)$.
**Time:** $\mathcal{O}(|\Sigma|N)$

8ad63e, 49 lines

```
struct AhoCorasick {
  enum {alpha = 26, first = 'a'}; // change this!
  struct Node {
    int par, link = -1, out = -1, to[alpha]{}, memo[alpha];
    char ch; ↑277649
    bool end = false;
    Node(int p = -1, char c = 0) : par(p), ch(c) {
      fill(memo, memo + alpha, -1);
    }
  }; ↑b08fea
  vector<Node> N{Node()};

  // insert s into the trie and return its last node
  int insert(string s) {
    int v = 0; ↑3732d3
    for (char c : s) {
      int i = c - first, u = N[v].to[i];
      if (!u) {
        u = sz(N);
        N[v].to[i] = u; ↑53dd48
        N.emplace_back(v, c);
      }
      v = u;
    } ↑7af61f
    N[v].end = true;
    return v;
  }
  // suffix link of node v
  int get_link(int v) {
    int& l = N[v].link; ↑d9b598
    if (l == -1) l = v && N[v].par ? go(get_link(N[v].par), N[v
        ].ch) : 0;
    return l;
  } ↑417806
  // from node v, follow the transition for character c
  int go(int v, char c) {
    int i = c - first, &g = N[v].memo[i];
    if (g == -1) g = N[v].to[i] ? N[v].to[i] : v ? go(get_link(
        v), c) : 0;
    return g;
  } ↑d18d66
  // longest proper suffix of node v among all patterns
  int next(int v) {
    int& u = N[v].out;
    if (u == -1) {
      int l = get_link(v);
```

```
      u = !l || N[l].end ? l : next(l);
    } ↑da6a0c
    return u;
  }
}; ↑8ad63e
```

# Various (10)

## 10.1 Intervals

### IntervalContainer.h

**Description:** Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, inclusive] over integers; adjacent intervals are merged.
**Time:** $\mathcal{O}(\log N)$ d99606, 23 lines

```
set<pii>::iterator addInterval(set<pii>& is, int L, int R) {
  if (L > R) return is.end();
  auto it = is.lower_bound({L, R}), before = it;
  while (it != is.end() && (ll)it->f <= (ll)R + 1) {
    R = max(R, it->s); ↑6b82ab
    before = it = is.erase(it);
  }
  if (it != is.begin() && (ll)(--it)->s + 1 >= L) {
    L = min(L, it->f);
    R = max(R, it->s); ↑07b135
    is.erase(it);
  }
  return is.insert(before, {L,R});
} ↑7d2bfc

void removeInterval(set<pii>& is, int L, int R) {
  if (L > R) return;
  auto it = addInterval(is, L, R);
  auto r2 = it->s;
  if (it->f == L) is.erase(it); ↑4beebe
  else { auto left = *it; is.erase(it); is.emplace(left.f, L-1)
       ; }
  if (R != r2) is.emplace(R+1, r2);
} ↑d99606
```

### IntervalCover.h

**Description:** Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add `|| R.empty()`. Returns empty set on failure (or if G is empty).
**Time:** $\mathcal{O}(N \log N)$ 97ff03, 19 lines

```
template<class T>
vi cover(pair<T, T> G, vector<pair<T, T>> I) {
  vi S(sz(I)), R;
  iota(all(S), 0); ↑fa5016
  sort(all(S), [&](int a, int b) { return I[a] < I[b]; });
  T cur = G.f;
  int at = 0;
  while (cur < G.s) { // (A)
    pair<T, int> mx = make_pair(cur, -1); ↑5ced66
    while (at < sz(I) && I[S[at]].f <= cur) {
      mx = max(mx, make_pair(I[S[at]].s, S[at]));
      at++;
    } ↑3583d4
    if (mx.s == -1) return {};
    cur = mx.f;
    R.push_back(mx.s);
  }
  return R;
} ↑97ff03
```

### ConstantIntervals.h

**Description:** Split a monotone function on [from, to] into a minimal set of inclusive intervals on which it has the same value. Runs a callback g for each such interval.
**Usage:** `constantIntervals(0, sz(v)-1, [&](int x){return v[x];}, [&](int lo, int hi, T val){...});`
**Time:** $\mathcal{O}\left(k \log \frac{n}{k}\right)$ 2f8ab2, 19 lines

```
template<class F, class G, class T>
void rec(int from, int to, F& f, G& g, int& i, T& p, T q) {
  if (p == q) return;
  if (from == to) {
    g(i, to-1, p); ↑69a8d8
    i = to; p = q;
  } else {
    int mid = (from + to) >> 1;
    rec(from, mid, f, g, i, p, f(mid));
    rec(mid+1, to, f, g, i, p, q);
  } ↑2c79e2
}
template<class F, class G>
void constantIntervals(int from, int to, F f, G g) {
  if (to < from) return; ↑026d40
  int i = from; auto p = f(i), q = f(to);
  rec(from, to, f, g, i, p, q);
  g(i, to, q);
} ↑2f8ab2
```

## 10.2 Misc. algorithms

### FastKnapsack.h

**Description:** Given $N$ non-negative integer weights $w$ and a non-negative target $t$, computes the maximum $S \leq t$ such that $S$ is the sum of some subset of the weights.
**Time:** $\mathcal{O}(N \max(w_i))$ b20ccc, 16 lines

```
int knapsack(vi w, int t) {
  int a = 0, b = 0, x;
  while (b < sz(w) && a + w[b] <= t) a += w[b++];
  if (b == sz(w)) return a;
  int m = *max_element(all(w)); ↑e2b1c9
  vi u, v(2*m, -1);
  v[a+m-t] = b;
  rep(i,b,sz(w)) {
    u = v;
    rep(x,0,m) v[x+w[i]] = max(v[x+w[i]], u[x]); ↑e898de
    for (x = 2*m; --x > m;) rep(j, max(0,u[x]), v[x])
      v[x-w[j]] = max(v[x-w[j]], j);
  }
  for (a = t; v[a+m-t] < 0; a--) ;
  return a;
} ↑b20ccc
```

### Random.h

**Description:** Better + seeded RNG 798de3, 1 lines

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().
    count());
```

## 10.3 Dynamic programming

### KnuthDP.h

**Description:** When doing DP on intervals: $a[i][j] = \min_{i<k<j}(a[i][k] + a[k][j]) + f(i,j)$, where the (minimal) optimal $k$ increases with both $i$ and $j$, one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j-1]$ and $p[i+1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b,c) \leq f(a,d)$ and $f(a,c) + f(b,d) \leq f(a,d) + f(b,c)$ for all $a \leq b \leq c \leq d$. Consider also: LineContainer (ch. Data structures), monotone queues, ternary search.
**Time:** $\mathcal{O}(N^2)$

### DivideAndConquerDP.h

**Description:** Given $a[i] = \min_{lo(i) \leq k < hi(i)}(f(i,k))$ where the (minimal) optimal $k$ increases with $i$, computes $a[i]$ for $i = L..R-1$.
**Time:** $\mathcal{O}\left((N + (hi - lo)) \log N\right)$ 1c65a0, 18 lines

```
struct DP { // Modify at will:
  int lo(int ind) { return 0; }
  int hi(int ind) { return ind; }
  ll f(int ind, int k) { return dp[ind][k]; } ↑29ea0c
  void store(int ind, int k, ll v) { res[ind] = pii(k, v); }

  void rec(int L, int R, int LO, int HI) {
    if (L >= R) return;
    int mid = (L + R) >> 1;
    pair<ll, int> best(LLONG_MAX, LO); ↑81b86d
    rep(k, max(LO,lo(mid)), min(HI,hi(mid)))
      best = min(best, make_pair(f(mid, k), k));
    store(mid, best.s, best.f);
    rec(L, mid, LO, best.s+1);
    rec(mid+1, R, best.s, HI); ↑0d2617
  }
  void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
}; ↑1c65a0
```

## 10.4 Debugging tricks

- `signal(SIGSEGV, [](int) { _Exit(0); });` converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). `_GLIBCXX_DEBUG` failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).
- `feenableexcept(29);` kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

## 10.5 Optimization tricks

`__builtin_ia32_ldmxcsr(40896);` disables denormals (which make floats 20x slower near their minimum value).

### 10.5.1 Bit hacks

- `x & -x` is the least bit in `x`.
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of `m` (except `m` itself).
- `c = x&-x, r = x+c; (((r^x) >> 2)/c) | r` is the next number after `x` with the same number of bits set.
- `rep(b,0,K) rep(i,0,(1 << K)) if (i & 1 << b) D[i] += D[i^(1 << b)];` computes all sums of subsets.

### 10.5.2 Pragmas

- `#pragma GCC optimize ("Ofast")` will make GCC auto-vectorize loops and optimizes floating points better.
- `#pragma GCC target ("avx2")` can double performance of vectorized code, but causes crashes on old machines.
- `#pragma GCC optimize ("trapv")` kills the program on integer overflows (but is really slow).

## Pragma.h

**Description:** cheese lol

293856, 3 lines

```
#include <vector>
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,popcnt,lzcnt,tune=native")
```

## FastMod.h

**Description:** Compute $a\%b$ about 5 times faster than usual, where $b$ is constant but not known at compile time. Returns a value congruent to $a$ (mod $b$) in the range $[0, 2b)$.

751a02, 8 lines

```
typedef unsigned long long ull;
struct FastMod {
  ull b, m;
  FastMod(ull b) : b(b), m(-1ULL / b) {} ↑551bab
  ull reduce(ull a) { // a % b + (0 or b)
    return a - (ull)((__uint128_t(m) * a) >> 64) * b;
  }
}; ↑751a02
```

## FastInput.h

**Description:** Read an integer from stdin. Usage requires your program to pipe in input from file.
**Usage:** `./a.out < input.txt`
**Time:** About 5x as fast as cin/scanf.

7b3c70, 17 lines

```
inline char gc() { // like getchar()
  static char buf[1 << 16];
  static size_t bc, be;
  if (bc >= be) {
    buf[0] = 0, bc = 0; ↑c5125f
    be = fread(buf, 1, sizeof(buf), stdin);
  }
  return buf[bc++]; // returns 0 on EOF
} ↑0261eb

int readInt() {
  int a, c;
  while ((a = gc()) < 40);
  if (a == '-') return -readInt();
  while ((c = gc()) >= 48) a = a * 10 + c - 480;
  return a - 48;
} ↑7b3c70
```

## BumpAllocator.h

**Description:** When you need to dynamically allocate many objects and don't care about freeing them. "new X" otherwise has an overhead of something like 0.05us + 16 bytes per allocation.

745db2, 8 lines

```
// Either globally or in a single class:
static char buf[450 << 20];
void* operator new(size_t s) {
  static size_t i = sizeof buf;
  assert(s < i); ↑c17d54
  return (void*)&buf[i -= s];
}
void operator delete(void*) {} ↑745db2
```

## SmallPtr.h

**Description:** A 32-bit pointer that points into BumpAllocator memory.

"BumpAllocator.h" 2dd6c9, 10 lines

```
template<class T> struct ptr {
  unsigned ind;
  ptr(T* p = 0) : ind(p ? unsigned((char*)p - buf) : 0) {
    assert(ind < sizeof buf);
  } ↑bda3ee
  T& operator*() const { return *(T*)(buf + ind); }
  T* operator->() const { return &**this; }
  T& operator[](int a) const { return (&**this)[a]; }
  explicit operator bool() const { return ind; }
}; ↑2dd6c9
```

## BumpAllocatorSTL.h

**Description:** BumpAllocator for STL containers.
**Usage:** `vector<vector<int, small<int>>> ed(N);`

bb66d4, 14 lines

```
char buf[450 << 20] alignas(16);
size_t buf_ind = sizeof buf;

template<class T> struct small {
  typedef T value_type; ↑2c8bf2
  small() {}
  template<class U> small(const U&) {}
  T* allocate(size_t n) {
    buf_ind -= n * sizeof(T);
    buf_ind &= 0 - alignof(T); ↑95ca9f
    return (T*)(buf + buf_ind);
  }
  void deallocate(T*, size_t) {}
}; ↑bb66d4
```

## Unrolling.h

520e76, 5 lines

```
#define F {...; ++i;}
int i = from;
while (i&3 && i < to) F // for alignment, if needed
while (i + 4 <= to) { F F F F }
while (i < to) F
```

## SIMD.h

**Description:** Cheat sheet of SSE/AVX intrinsics, for doing arithmetic on several numbers at once. Can provide a constant factor improvement of about 4, orthogonal to loop unrolling. Operations follow the pattern `"_mm(256)?_name_(si(128|256)|epi(8|16|32|64)|pd|ps)"`. Not all are described here; grep for `_mm_` in `/usr/lib/gcc/*/4.9/include/` for more. If AVX is unsupported, try 128-bit operations, "emmintrin.h" and `#define __SSE__` and `__MMX__` before including it. For aligned memory use `_mm_malloc(size, 32)` or `int buf[N] alignas(32)`, but prefer loadu/storeu.

c9ac08, 43 lines

```
#pragma GCC target ("avx2") // or sse4.1
#include "immintrin.h"

typedef __m256i mi; ↑bb95ae
#define L(x) _mm256_loadu_si256((mi*)&(x))

// High-level/specific methods:
// load(u)?_si256, store(u)?_si256, setzero_si256, _mm_malloc
// blendv_(epi8|ps|pd) (z?y:x), movemask_epi8 (hibits of bytes)
// i32gather_epi32(addr, x, 4): map addr[] over 32-b parts of x
// sad_epu8: sum of absolute differences of u8, outputs 4xi64
// maddubs_epi16: dot product of unsigned i7's, outputs 16xi15
// madd_epi16: dot product of signed i16's, outputs 8xi32
// extractf128_si256(, i) (256->128), cvtsi128_si32 (128->lo32)
// permute2f128_si256(x,x,1) swaps 128-bit lanes
// shuffle_epi32(x, 3*64+2*16+1*4+0) == x for each lane
// shuffle_epi8(x, y) takes a vector instead of an imm

// Methods that work with most data types (append e.g. _epi32):
// set1, blend (i8?x:y), add, adds (sat.), mullo, sub, and/or,
// andnot, abs, min, max, sign(1,x), cmp(gt|eq), unpack(lo|hi)

int sumi32(mi m) { union {int v[8]; mi m;} u; u.m = m;
  int ret = 0; rep(i,0,8) ret += u.v[i]; return ret; }
mi zero() { return _mm256_setzero_si256(); } ↑5b5012
mi one() { return _mm256_set1_epi32(-1); }
bool all_zero(mi m) { return _mm256_testz_si256(m, m); }
bool all_one(mi m) { return _mm256_testc_si256(m, one()); }

ll example_filteredDotProduct(int n, short* a, short* b) {
  int i = 0; ll r = 0; ↑9a33eb
  mi zero = _mm256_setzero_si256(), acc = zero;
  while (i + 16 <= n) {
    mi va = L(a[i]), vb = L(b[i]); i += 16; ↑24ffdb
    va = _mm256_and_si256(_mm256_cmpgt_epi16(vb, va), va);
    mi vp = _mm256_madd_epi16(va, vb);
    acc = _mm256_add_epi64(_mm256_unpacklo_epi32(vp, zero),
      _mm256_add_epi64(acc, _mm256_unpackhi_epi32(vp, zero)));
  } ↑3f47f6
  union {ll v[4]; mi m;} u; u.m = acc; rep(i,0,4) r += u.v[i];
  for (;i<n;++i) if (a[i] < b[i]) r += a[i]*b[i]; // <- equiv
  return r;
} ↑c9ac08
```

| | |
|---|---|
| you just do it. | I can't explain the solution |
| don't you just do it? | I have the first step |
| why isn't this trivial? | I have what I think is most of the problem |
| it's standard but I don't know how to do it | it's nonstandard and I don't know how to do it |
| I have the correct AC idea | I can explain the idea |
| I have the correct WA idea | I have a proof by AC idea |
| it's convex | it's not convex |
| it's subfake | the proof by AC is WA |
| disjoint cycle | observation farm |
| do you want to submit bullshit randomized cheese to this problem? | if nobody else is typing, I'm typing |
| epsilon cheese | replace 1e-9 with epsilon and try random values of epsilon until it ACs |
| it's trivial trivial trivial | give me the computer |



# Techniques (A)

## techniques.txt

159 lines

```
Recursion
Divide and conquer
  Finding interesting points in N log N
Algorithm analysis
  Master theorem
  Amortized time complexity
Greedy algorithm
  Scheduling
  Max contiguous subvector sum
  Invariants
  Huffman encoding
Graph theory
  Dynamic graphs (extra book-keeping)
  Breadth first search
  Depth first search
  * Normal trees / DFS trees
  Dijkstra's algorithm
  MST: Prim's algorithm
  Bellman-Ford
  Konig's theorem and vertex cover
  Min-cost max flow
  Lovasz toggle
  Matrix tree theorem
  Maximal matching, general graphs
  Hopcroft-Karp
  Hall's marriage theorem
  Graphical sequences
  Floyd-Warshall
  Euler cycles
  Flow networks
  * Augmenting paths
  * Edmonds-Karp
  Bipartite matching
  Min. path cover
  Topological sorting
  Strongly connected components
  2-SAT
  Cut vertices, cut-edges and biconnected components
  Edge coloring
  * Trees
  Vertex coloring
  * Bipartite graphs (=> trees)
  * 3^n (special case of set cover)
  Diameter and centroid
  K'th shortest path
  Shortest cycle
Dynamic programming
  Knapsack
  Coin change
  Longest common subsequence
  Longest increasing subsequence
  Number of paths in a dag
  Shortest path in a dag
  Dynprog over intervals
  Dynprog over subsets
  Dynprog over probabilities
  Dynprog over trees
  3^n set cover
  Divide and conquer
  Knuth optimization
  Convex hull optimizations
  RMQ (sparse table a.k.a 2^k-jumps)
  Bitonic cycle
  Log partitioning (loop over most restricted)
Combinatorics
  Computation of binomial coefficients
  Pigeon-hole principle
  Inclusion/exclusion
  Catalan number
  Pick's theorem
Number theory
  Integer parts
  Divisibility
  Euclidean algorithm
  Modular arithmetic
  * Modular multiplication
  * Modular inverses
  * Modular exponentiation by squaring
  Chinese remainder theorem
  Fermat's little theorem
  Euler's theorem
  Phi function
  Frobenius number
  Quadratic reciprocity
  Pollard-Rho
  Miller-Rabin
  Hensel lifting
  Vieta root jumping
Game theory
  Combinatorial games
  Game trees
  Mini-max
  Nim
  Games on graphs
  Games on graphs with loops
  Grundy numbers
  Bipartite games without repetition
  General games without repetition
  Alpha-beta pruning
Probability theory
Optimization
  Binary search
  Ternary search
  Unimodality and convex functions
  Binary search on derivative
Numerical methods
  Numeric integration
  Newton's method
  Root-finding with binary/ternary search
  Golden section search
Matrices
  Gaussian elimination
  Exponentiation by squaring
Sorting
  Radix sort
Geometry
  Coordinates and vectors
  * Cross product
  * Scalar product
  Convex hull
  Polygon cut
  Closest pair
  Coordinate-compression
  Quadtrees
  KD-trees
  All segment-segment intersection
Sweeping
  Discretization (convert to events and sweep)
  Angle sweeping
  Line sweeping
  Discrete second derivatives
Strings
  Longest common substring
  Palindrome subsequences
  Knuth-Morris-Pratt
  Tries
  Rolling polynomial hashes
  Suffix array
  Suffix tree
  Aho-Corasick
  Manacher's algorithm
  Letter position lists
Combinatorial search
  Meet in the middle
  Brute-force with pruning
  Best-first (A*)
  Bidirectional search
  Iterative deepening DFS / A*
Data structures
  LCA (2^k-jumps in trees in general)
  Pull/push-technique on trees
  Heavy-light decomposition
  Centroid decomposition
  Lazy propagation
  Self-balancing trees
  Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)
  Monotone queues / monotone stacks / sliding queues
  Sliding queue using 2 stacks
  Persistent segment tree
```

## dress-rehearsal.txt

33 lines

```
Keyboard layouts
template.cpp
samps.sh
Java (maybe not)
Python
Printing
Sending clarifications
Documentation
Submit script
Whitespace/case insensitivity in output
Return values from main
Printing to stderr
Source code limit
Is it possible to submit and read from non-source files?
SIMD support on machine resp. judge
How long do array accesses take on machine resp. judge?
How long do small operations take on machine resp. judge?
Does UBSan work? Does ASan work?
All judge errors:
  Accepted
  WA
  Presentation Error
  RTE (what results in RTE?)
  TLE
  Memory limit, both stack and heap
  Output limit
  Illegal function
  Compile error
  Compile time limit exceeded
  Compile memory limit exceeded
  Too late
Listing all available binaries
  echo $PATH | tr ':' ' ' | xargs ls | grep -v / | sort | uniq
       | tr 'char`\n' ' ' > path.txt
```